IMPLEMENTATION OF AGV FOR HOSPITAL LOGISTICS

AYSER AMIR KAMIL MOHAMED (TP067377)
BADR BASSAM HASSAN ALBDULMAJEED (TP061632)
ISAMELDIN AHMED HUSSEIN AHMED (TP069561)
RAYAN KHALID MUBARAK AL YAQOUBI (TP063236)

DR. SATHISH KUMAR SELVAPERUMAL

ASIA PACIFIC UNIVERSITY OF TECHNOLOGY & INNOVATION
SCHOOL OF ENGINEERING

Jan 2026

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

SYMBOL, ABBREVIATION OR NOMENCLATURE	FULL MEANING
2D	Two-Dimensional
3D	Three-Dimensional
4D	Four-Dimensional
A*	A-Star Algorithm
AGV	Automated Guided Vehicle
AI	Artificial Intelligence
API	Application Programming Interface
BEM	Board of Engineers Malaysia
BOM	Bill of Materials
CAD	Computer-Aided Design
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
DC	Direct Current
DEA	Data Envelopment Analysis
DES	Discrete Event Simulation
ESP32	Espressif Systems 32-bit Microcontroller
FPS	Frames Per Second
GDP	Group Design Project
GHz	Gigahertz
GPS	Global Positioning System
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HUSM	Hospital Universiti Sains Malaysia
Hz	Hertz
IDE	Integrated Development Environment
IoT	Internet of Things
IR	Infrared
kg	Kilogram
LiDAR	Light Detection and Ranging

LIRC	Linux Infrared Remote Control
LTS	Long Term Support
mAP	Mean Average Precision
mAP50	Mean Average Precision at 50% IoU
mAP50-95	Mean Average Precision from 50% to 95% IoU
MBTI	Myers-Briggs Type Indicator
mm	Millimeter
NPV	Net Present Value
NVCC	NVIDIA CUDA Compiler
OCR	Optical Character Recognition
ONNX	Open Neural Network Exchange
ORB	Oriented FAST and Rotated BRIEF
OS	Operating System
PLC	Programmable Logic Controller
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
PWM	Pulse Width Modulation
PyQt5	Python Qt5 Framework
PyTorch	Python Machine Learning Library
RGB	Red Green Blue
RGB-D	Red Green Blue - Depth
RFID	Radio Frequency Identification
RM	Ringgit Malaysia
ROI	Return on Investment
ROS	Robot Operating System
ROS2	Robot Operating System 2
SCM	Supply Chain Management
SDK	Software Development Kit
SLAM	Simultaneous Localization and Mapping
SPI	Serial Peripheral Interface
TEB	Time Elastic Band
UID	Unique Identifier
USB	Universal Serial Bus
V	Volt
W	Watt
Wh	Watt-hour

WiFi	Wireless Fidelity
YOLO	You Only Look Once
YOLOv3	You Only Look Once Version 3
YOLOv5	You Only Look Once Version 5
YOLOv8	You Only Look Once Version 8
YOLOv11	You Only Look Once Version 11
YOLOv12	You Only Look Once Version 12

CHAPTER 1 INTRODUCTION TO STUDY

1.1 Introduction

Hospitals have to move medicine, food, and equipment to patients every day and staff spend time on manual labor. Research shows that nurses and staff spend about 2.5 hours per shift just moving things around (Pedan, Gregor, & Plinta, 2017). This takes time away from caring for patients, additionally things could get lost or arrive late because the staff is too busy.

AGVs can fix this, because it's a robot that moves by itself w autonomously. Additionally, it can carry items safely from place to place and it can run most task independently regardless the time of the day, unless it's charging. (Gargouri, Karray, Zalila, & Ksantini, 2025).

Hospitals in other countries implement AGVs to assist in delivering food, medicine, and laundry to patients (Pedan et al., 2017). These robots do the physical work, so that staff can focus on patients instead of moving carts. However, AGVs cost more money and need to be safe around people (Medjaldi, Slimani, & Karkar, 2025).

This project aims to builds an AGV for hospital use, it will use cameras, lidars and sensors to detect obstacles and avoid them (Duhan & Panwar, 2024). Moreover, it will plan and navigate safe paths to move supplies. The goal of this project is to develop a robot that is reliable and cost effective to implement.

Furthermore, A study done by (Zakaria, Zakaria, Rassip, & Lee, 2022) found that 24.4% of Malaysian nurses experience burn out from too much work in hospitals and could use this technology to reduce their workload especially when they don't have enough staff and need to be cost efficient. An AGV system would help hospitals run better and make taking care of patients easier for the staff.

1.2 Research Problems

1.2.1 Staff Workload and Physical Strain

Nurses and staff push carts for about 2.5 hours per shift (Pedan et al., 2017). They deliver food, move supplies, and pick up trash, all this physical labour buts a strain their bodies. A study done on Malaysian hospitals found that this type of

physical work is one of the main reasons why nurses face burnout and become more susceptible to making mistakes (Zakaria et al., 2022).

1.2.2 Time Away from Patient Care

Research shows that nurses only spend 30-40% of their shift caring for patients (Pedan et al., 2017); while the of the time is spent on paperwork and moving supplies, so if the transport work is automated using AGVs, nurses can spend that time with the patients instead.

1.2.3 Safety and Infection Control

Pushing carts isn't always safe. Staff can't see around corners when pushing big carts, so they crash into people. Also, biohazards waste needs careful handling so diseases don't spread (Zakaria et al., 2022). Humans can forget or they don't always follow safety rules; however, in an AGVs can follow safe paths, avoid obstacles and safely carry biohazard wastes.

1.2.4 Delivery Errors and Delays

A study in a Malaysian hospital found a 30.5% error rate for medicine in the emergency room (Shitu, Aung, Tuan Kamauzaman, & Ab Rahman, 2020) Because people make mistakes and sometimes the medicine gets delivered to the wrong patient in the wrong room or in some cases delays happen when staff get busy; thus automated systems with tracking systems can stop these errors or delays.

1.2.5 Cost of Labor

AGVs save money in the long run because they cut labor costs and work more efficiently. Paying staff is expensive and it’s a waste to have trained medical staff pushing carts instead of caring for their patients. Research has found that logistics take up to 40% of hospital costs (Pedan et al., 2017).

1.2.6 Scalability Issues

Logistics become difficult as hospitals grow larger, because having more floors means longer walking distances. Moreover, manual labor is not effective

for large areas (Gargouri et al., 2025). Furthermore, hiring more staff increases the costs. Therefore, implementing an AGV system will be beneficial for hospitals.

1.3 Aim and Objectives

Aim:

The aim of this project is to design and build a working prototype of a smart AGV that can securely carry medical supplies and biohazard waste inside a hospital.

Objectives:

1. To develop an autonomous navigation system.
2. To implement a secure dual-payload compartment system accessible only via RFID authentication.
3. To design a safety system that automatically stops the vehicle when obstacles are detected.
4. To test the AGV system implantation.

1.4 Justification for This Research

1.4.1 Addressing Healthcare Challenges

The number of nurses in Malaysia is lower than what is needed for adequate patient care, which result in Malaysian nurses being exhausted because their workload (Zakaria et al., 2022). So any technology that lowers their workload is useful because it lets them focus on patient care.

1.4.2 Technological Advancement

Robotics and electronics have improved a lot in recent years. It is easier to builds an AGV because parts are cheaper and the software is more accessible. New AI tools like YOLOv8 can effectively and efficiently detect objects with over 92% accuracy (Duhan & Panwar, 2024); making it possible for students to build working AGV prototypes (Gargouri et al., 2025).

1.4.3 Knowledge Gap

In hospitals hallways are narrow and people walk in random ways, most of the AGV applications are at factories or warehouses. But hospitals are different (Pedan et al., 2017). Additionally, hospitals have stricter safety rules, so this project focuses only on hospitals to fill this gap in this technology.

1.4.4 Economic Impact

This technology could be a cost saving measure for hospitals as research had shown that a huge percentage of a hospital's budget goes into material transport (Pedan et al., 2017). Buying AGVs costs a lot at first; however, they save money in the long run because they cut labour costs and work more effectively.

1.4.5 Potential for Commercialization

A working robot could become a product to sell, because Malaysian hospitals might start using them because making them locally costs less than buying them from other countries (Medjaldi et al., 2025).

1.5 Organisation for The Rest of The Chapters

The rest of this report is organized into nine chapters. These chapters explain the design, build, and testing of the AGV.

- Chapter 2 covers the machine vision subsystem, which explains the selection of the Nvidia Jetson Orin Nano and ZED 2i camera. Moreover, it details the training process of the YOLOv8 AI models to detect obstacles and medicines.
- Chapter 3 focuses on the electrical power and motor control system, it describes how the 48V battery powers the AGV. Furthermore, it explains the differential steering logic using the ESP32 microcontroller.
- Chapter 4 describes the mechanical design and navigation covering the chassis construction and suspension system. Additionally, it explains autonomous path planning using the Unitree 4D LiDAR and ROS2.
- Chapter 5 writing is about the security and IoT monitoring system and the RFID access control for the payload compartments. Moreover, it explains the remote dashboard built with Blynk.

- Chapter 6 shows the integrated system and describes how all hardware and software parts were connected. Furthermore, it presents the final test results for the complete AGV.
- Chapter 7 discusses professional engineering practices and it covers the safety, legal, and cultural standards.
- Chapter 8 covers sustainability and analyzes the environmental impact and economic benefits of the AGV.
- Chapter 9 outlines project management. It includes the Gantt chart used for planning and the final financial budget.
- Chapter 10 discusses the discussion and conclusion parts which summarizes the project achievements, limitations, and recommendations for future improvements.

1.6 Summary

In summary, this chapter sets the groundwork for the project as it introduces the problems Malaysian hospitals face with manual logistics, labor exhaustion, and cost. Moreover, it defines the problem the team found and why it matters and justifies why this research is important. Furthermore, it explains the aims and objectives for the AGV. Finally, it provides a roadmap of the report organization.

CHAPTER 2 BADR ALBDULMAJEED TP061632

2.1 Introduction

The healthcare industry is using new technology to fix its problems and recent papers show that automation is the best way to handle logistics issues. Moreover, studies prove that AGVs can lower costs as Pedan et al. (2017) found that logistics make up 40% of hospital expenses. Therefore, hospitals can save money by automating these tasks. Furthermore, automation makes moving safer. Reducing human contact is very important during pandemics like COVID-19. Robots can travel through infectious areas without getting sick. Thus, this protects the staff from hospital-acquired infections.

2.2 Literature review

2.2.1 Comparative Analysis of Object Detection Algorithms

A big challenge for this robot is seeing its surroundings, because the robot must know if the object in front is a wall, a person, or a bed. Therefore, it needs a computer vision algorithm. The team looked at several options to pick the best one. Furthermore, Duhan and Panwar (2024) did a detailed study comparing versions of YOLO algorithms in hospitals.

Table 2.1: Comparative Analysis of YOLO Architectures

Model Version	Detection Accuracy (mAP)	Inference Speed	Suitability for Hospitals
YOLOv3	78.4%	Slow (10 FPS)	Poor, because this model is outdated & slow for a moving robot to react safely to people.
YOLOv5	85.1%	Moderate (20 FPS)	Average but widely used, it struggles to detect small items like medicine bottles in dim light.
YOLOv8	92.5%	Fast (30 FPS)	Excellent since it offers the highest accuracy for small objects and works well in low-light conditions (night shifts).
YOLOv11	90.0%	Very Fast (40 FPS)	Good but Risky, because faster, lightweight versions often trade off recall,

			potentially missing small obstacles.
YOLOv12	94.0%	Slow on Edge (15 FPS)	Excellent but requires too much computational power not available on the Jetson Nano, which cause latency.

Based on the data in Table 2.1 the chosen model is YOLOv8, since it has the highest accuracy of 92.5%. Besides, it can maintain performance (Duhan & Panwar, 2024). This is important because a hospital robot cannot make mistakes. Moreover, YOLOv8 works well in low light, which is an advantage because hospitals operate at night and the lights might be dim in some corridors.

2.2.2 Hardware and Embedded Computing

Running AI models needs a strong computer. However, a standard laptop is too big. Besides, it uses too much power for a mobile robot. Therefore a small, embedded computer was chosen.

Gargouri et al. (2025) tested a Raspberry Pi 4 for a similar robot. However, they faced problems; their study showed that the Raspberry Pi was too slow. Furthermore, it struggled to run the navigation software and AI vision together, as a result, the robot lagged and reacted slowly to obstacles.

To avoid this failure, this project selected the NVIDIA Jetson Orin Nano it's different from the Raspberry Pi. Moreover, it has a dedicated GPU which acts as the brain for AI tasks.

Table 2.2: Hardware Performance Comparison – Jetson Orin Nano vs. Raspberry Pi 4

Feature	**Raspberry Pi 4 (Reference Study)**	**NVIDIA Jetson Orin Nano**
Processor Type	CPU Only	GPU Accelerated (1024 CUDA Cores)
AI Performance	Low & struggles with modern AI	Designed specifically for AI
Multitasking	Failed to run SLAM + AI smoothly	Runs ROS2 + YOLOv8 simultaneously
Suitability	Hobbyist Projects	Professional Robotics

Table 2.2 shows why the Jetson is better. Besides, the GPU lets the robot process images very fast. Moreover, it does not slow down the navigation.

Therefore, this decision makes sure the system is safe and reacts quickly (Gargouri et al., 2025).

Here is the paraphrased text following your rules and the sample style:

2.2.3 Sensor Fusion Strategy (RGB vs. RGB-D)

A regular camera (RGB) has a major weakness tt can identify an object, but it cannot tell how far away the object is. Therefore, this is dangerous for a robot. The robot might crash if it thinks a person is far away when they are close. Medjaldi et al. (2025) suggested a solution called Sensor Fusion their research showed that combining a camera with a depth sensor (RGB-D) is much safer. Moreover, the "D" stands for Depth which measures the exact distance to every pixel. Furthermore, their robot achieved a 95% success rate in avoiding obstacles. Based on this evidence, the ZED 2i Stereo Camera was chosen. This camera has two lenses to calculate depth accurately. Therefore, this lets the robot stop exactly 1 meter away from an obstacle. Finally, this ensures the safety of patients and staff (Medjaldi et al., 2025).

2.2.4 Software Development Environment

Building a robot needs a strong software, so the following table shows the specific software libraries and versions used. Furthermore, it lists the tools used to develop the autonomous navigation and medication systems on the Nvidia Jetson Orin Nano.

Table 2.3: Software Libraries and Environment Configuration

Component	Library/Tool	Version
Operating System	Ubuntu Linux	22.04.5 LTS
Jetson SDK	JetPack (L4T)	R36, Revision 4.7
Kernel	Linux Kernel	5.15.148-tegra
Programming Language	Python	3.10.12
GPU Acceleration	CUDA	12.6.68
Deep Learning Framework	PyTorch	2.8.0+cu126
Computer Vision	OpenCV (Headless)	4.12.0.88
AI Vision Library	Ultralytics YOLO	8.3.241
Inference Engine	ONNX Runtime GPU	1.23.0
Camera SDK	ZED SDK (PyZED)	5.1
GUI Framework	PyQt5	5.15.6
Numerical Computing	NumPy	1.26.4
Compiler	NVCC	12.6.68

To ensure the system was set up correctly for AI computing, the environment was checked, which was done using terminal commands on the Jetson Orin Nano. This step confirms that the hardware acceleration (CUDA) is active. Moreover, it confirms it is accessible to the Python scripts.

The command cat /etc/os-release confirms the OS is Ubuntu 22.04 LTS. Furthermore, the CUDA compiler version is verified as 12.6.68 using /usr/local/cuda/bin/nvcc --version. Most importantly, the Python script execution confirms PyTorch 2.8.0 (CUDA = True). Additionally, it confirms ONNX Runtime (Device = GPU). Therefore, this proves that the AI models are running on the GPU. Thus, this is necessary for real-time performance.

2.2.5 Vision Sensor Selection

Table 2.4: Comparison of Vision Sensors

Feature	**Standard Webcam (e.g., Logitech C920)**	**ZED 2i Stereo Camera (Selected)**
Depth Perception	None (2D image only)	Full 3D (Stereo Vision up to 20m)
Object Distance	Cannot measure distance	Measures distance with <1% error
Field of View	Narrow (78 degrees)	Wide (110 degrees)
Safety	Low safety	High safety for navigation

As shown in Table 2.4, the ZED 2i is much better for navigation tasks. Moreover, it provides the Depth data that a standard webcam lacks. Therefore, this sensor was chosen for the project.

Table 2.5: Summary of Literature Review

No.	Year of Publication	Authors	Methodology	Outcomes/Advantages
1	2024	Duhan & Panwar	Compared YOLOv3, v5, v8, v11, v12 in hospital settings	YOLOv8 achieved 92.5% mAP with 30 FPS; best for real-time hospital AGV
2	2025	Gargouri et al.	Tested Raspberry Pi 4 for autonomous navigation	Pi 4 failed to run SLAM + AI smoothly; proved need for GPU-accelerated hardware
3	2025	Medjaldi et al.	Compared RGB vs RGB-D sensors for AGV obstacle avoidance	RGB-D sensors achieved 95% obstacle avoidance success; depth sensing is critical

2.3 Investigation on Materials & Components Selection

The vision and safety subsystem depends on choosing the right electronic components and it must support real-time AI while staying compact enough to mount on the AGV.

First, different embedded boards were compared, such as Raspberry Pi 4 and Jetson Orin Nano. As shown in Table 2.2, the Raspberry Pi 4 struggles to run SLAM and AI at the same time, which means slow reaction to obstacles (Gargouri et al., 2025). Therefore, it was rejected for this project. On the hand, the Jetson Orin Nano was selected because it has a dedicated GPU and is designed for AI work tasks, and this can allow YOLOv8 and ROS2 to run together without frame drops.

Second, as for the camera options it was a standard USB webcam, such as the Logitech C920 which is cheap and easy to use, or a camera with depth sensors like ZED 2i stereo camera. The problem with the standard USB webcam that it only gives 2D RGB images and cannot measure distance. Meanwhile, Medjaldi et al. (2025) showed that combining RGB with depth sensors gives a 95% success rate in obstacle avoidance. Therefore, the ZED 2i stereo camera was chosen, as summarised in Table 2.4. It provides both RGB and depth, with distance error under 1%, which is important for safe stopping distance.

Third, depth sensing choices were a Kinect-style RGB-D sensor, which is cheaper than the zed 2i, but bulky and not designed for mobile robots in narrow hospital corridors (Medjaldi et al., 2025). Or the ZED 2i which is more compact and has a wider field of view, so it can see more of the corridor at once. This reduces blind spots near corners and improves safety.

Finally, the software libraries are listed in Table 2.3, so the Jetson Orin Nano officially supports JetPack, CUDA, PyTorch, and the ZED SDK, which reduces integration risk. Therefore, the final hardware used for this individual part consists of the Jetson Orin Nano, ZED 2i stereo camera, and the existing 4d lidar which the AI data supplied through ROS2 from the group navigation task (Gargouri et al., 2025; Medjaldi et al., 2025).

2.4 Methodology

The vision system uses deep learning to find obstacles and medicine. Moreover, has six main stages: dataset acquisition, pre-processing, training, validation, deployment, and decision making.

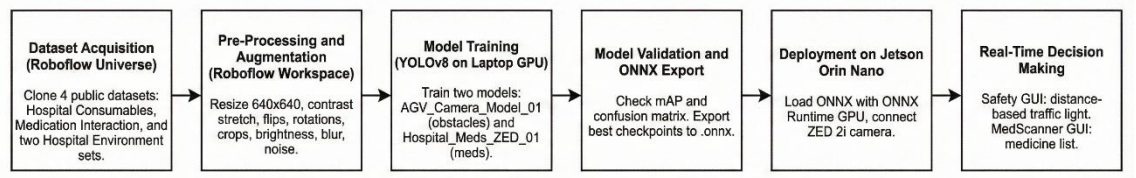


Figure 2.1: Methodology

2.4.1 Data Acquisition

The datasets were not made from scratch four existing projects from Roboflow Universe were used to make the machine vision training set.

For medication recognition, two public datasets were used:

- Hospital Consumables: This contains boxes, gloves, and bins (Skripsie-ghhbu, n.d.).
- Medication Interaction: This focuses on pills and drug packages (Test-project-csgdb, n.d.).

For obstacle detection, two hospital scene datasets were used:

- Hospital: This includes beds, trolleys, and doors in corridors (Hospital-3l55y, n.d.).
- Hospital: This is a second dataset with similar items to add variety (Logical-9iucs, n.d.).

These four datasets were cloned into a Roboflow workspace, then the classes' annotations were checked to ensure that the same objects use the same label name. After this, two internal projects were created:

- AGV_Camera_Model_01 for navigation safety.

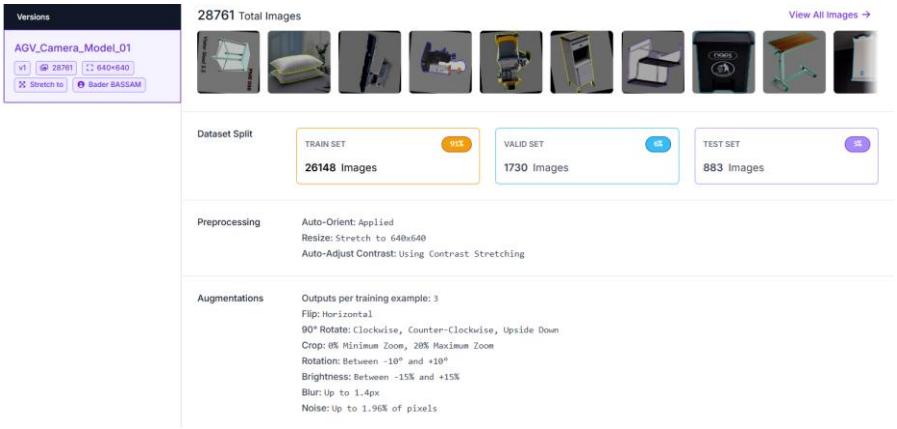


Figure 2.2: Navigation dataset roboflow

- Hospital_Meds_ZED_01 for medicine recognition.

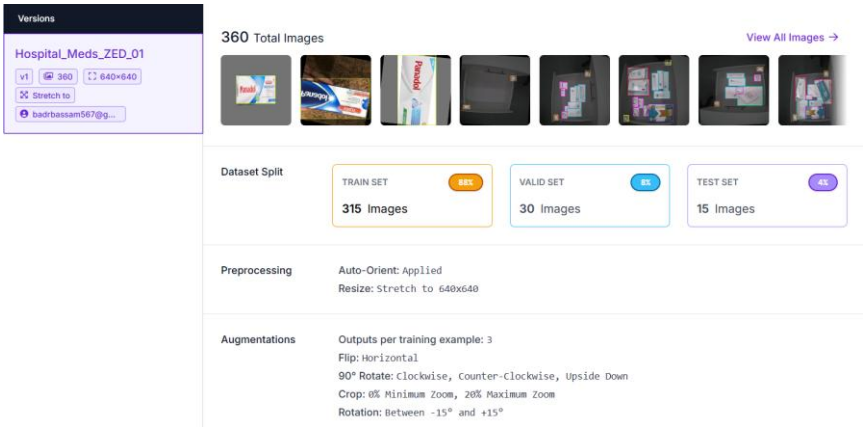


Figure 2.3: Medicine Dataset roboflow

2.4.2 Pre-Processing and Data Augmentation

- All images were resized to 640 × 640 pixels.
- Auto Orient makes the images have the right rotation.
- Contrast was changed to help see in dim lights.
- Augmentation of each image produced three new versions.
- Horizontal flips and 90° rotations were used.
- Random zooms, brightness changes, and noise were added.

These steps make the models strong against different angles and lighting, which should simulate a hospital environment.

2.4.3 Model Training Strategy

Both models use YOLOv8 as the base (Duhan & Panwar, 2024). However, the training was done on a laptop with a GPU, while the Jetson was used for the final GUIs.

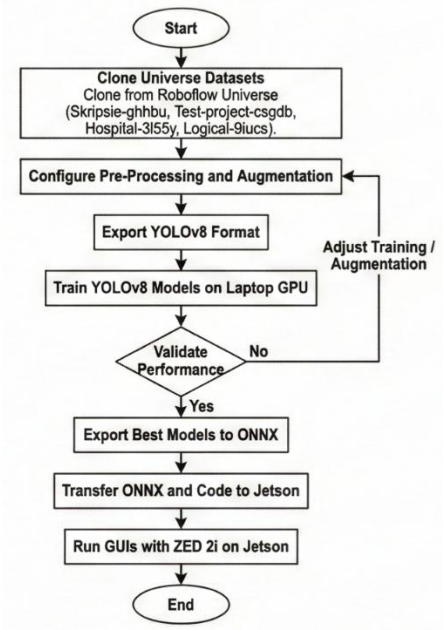


Figure 2.4: Training & Deployment Flow Chart

For each dataset:

- The split for training and testing followed the Roboflow settings.
- The YOLOv8 files were exported from Roboflow.
- Training ran for many epochs until the error was low.
- Finally, the best file was saved as ONNX. This format works well on the Jetson.

2.4.4 Distance Logic and Safety Thresholds

The Navigation GUI, which was built using python on pycharm IDE, uses the trained obstacle detection model with the ZED 2i depth camera. Moreover, the system uses the ZED 2i to make a depth map, so it checks the distance at the center of the object. Therefore, the safety state is set using three rules:

- Green (Safe): Distance is more than 2.0 m.
- Orange (Slow Down): Distance is between 1.0 m and 2.0 m.
- Red (Emergency Stop): Distance is less than 1.0 m.

2.5 Concept Design Derived from Fundamental Engineering Principles

The concept design of the vision system covers three parts: datasets and model training, hardware architecture, and the software. This part of the design is important because the final tasks of the AGV depends on how the data, models, and hardware work together. Therefore, the goal is to build a system that is safe, reliable, and simple to maintain.

For the data and model design, four public datasets from Roboflow Universe were chosen; these include two hospital datasets for obstacles and two medical datasets for medicines. Moreover, these were cloned in a roboflow workspace, then the annotations were checked and the next step is the pre-processing was and augmentations were done. So all images are resized to 640×640, enhanced with contrast, flips, rotations, and noise. This forms two internal datasets: AGV_Camera_Model_01 for obstacles and Hospital_Meds_ZED_01 for medicines. Finally, both datasets are used to train based on YOLOv8 models using Pycharm on a laptop GPU. Finally, The best files are exported to ONNX format to run on the Jetson.

For the hardware design, the system uses the ZED 2i stereo camera which captures RGB images and depth maps. Moreover, it connects via USB Type-C to the USB 3.0 port of the NVIDIA Jetson Orin Nano, and this allows data to transfered with low latency. Furthermore, the Jetson acts as the brain of the AGV, i runs Ubuntu with JetPack, in addition to some libraries such as, CUDA, and ONNX Runtime.

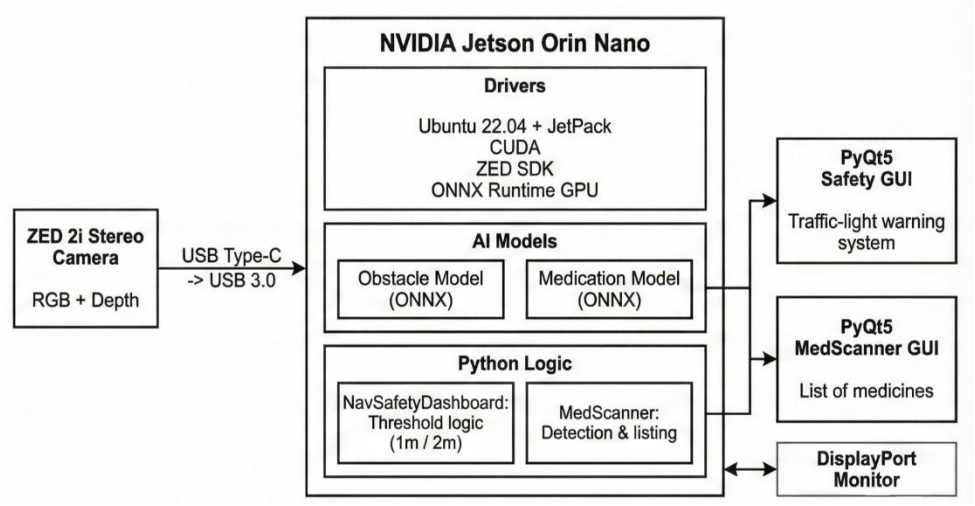


Figure 2.5: System architecture

For the software architecture, the design follows a modular structure. At the platform layer, drivers provide access to the GPU. Above this, the two ONNX models sit in an AI model layer. One is for obstacle detection, and one is for medication. On top of that, a Python layer contains two main classes. NavSafetyDashboard loads the obstacle model and grabs frames from the ZED camera. Then, it reads the depth value and applies three safety thresholds. MedScanner loads the medication model and detects medicines. Furthermore, it stores unique detections in a list to avoid duplicates.

This architecture represents the concept design. The data choices are planned so each model specializes in one task. Moreover, the hardware is chosen to support GPU inference. Furthermore, the software is split into modules that are easy to test. Finally, safety is prioritized by giving the NavSafetyDashboard a central role.

2.6 System Implementation

The system implementation follows four stages: training the datasets on a laptop, validating results for the trained models, testing GUIs on a PC with the ZED 2i camera, and final deployment on the Jetson Orin Nano.

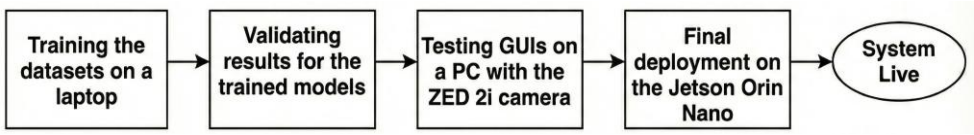


Figure 2.6: system Implementation flowchart

2.6.1 Training Environment and Libraries

The training was done on a laptop running Windows 11 with Python 3.11 in PyCharm, the libraries used are shown in Table 2.6.

Table 2.6: Python Libraries for Training

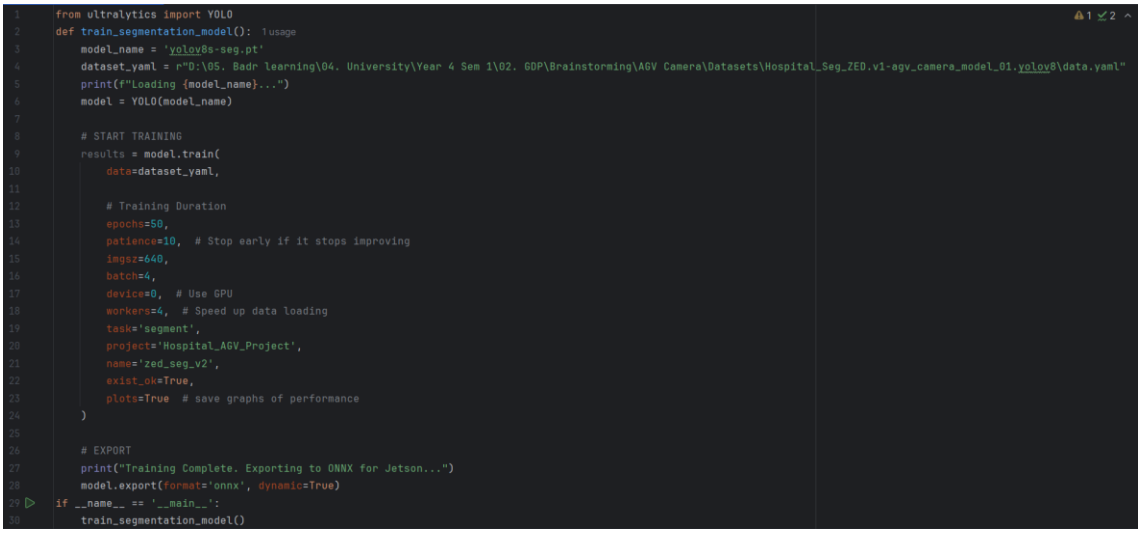
Package	Version
ultralytics	8.3.235
torch	2.5.1+cu121
torchvision	0.20.1+cu121
onnx	1.19.1
onnxruntime-gpu	1.23.2

opencv-python	4.12.0.88
numpy	2.2.6
pandas	2.3.3
matplotlib	3.10.8
pillow	12.0.0
pyzed	(SDK installer)

2.6.2 Model Training Scripts

Two separate training scripts were created for the two tasks. The training process uses Ultralytics YOLOv8 with early stopping to prevent overfitting and export to ONNX format for Jetson deployment.

1. Obstacle Detection Training Script:



```python
from ultralytics import YOLO
def train_segmentation_model():  1 usage
    model_name = 'yolov8s-seg.pt'
    dataset_yaml = r"D:\05. Badr learning\04. University\Year 4 Sem 1\02. GDP\Brainstorming\AGV Camera\Datasets\Hospital_Seg_ZED.v1-agv_camera_model_01.yolov8\data.yaml"
    print(f"Loading {model_name}...")
    model = YOLO(model_name)

    # START TRAINING
    results = model.train(
        data=dataset_yaml,

        # Training Duration
        epochs=50,
        patience=10,  # Stop early if it stops improving
        imgsz=640,
        batch=4,
        device=0,  # Use GPU
        workers=4,  # Speed up data loading
        task='segment',
        project='Hospital_AGV_Project',
        name='zed_seg_v2',
        exist_ok=True,
        plots=True  # save graphs of performance
    )

    # EXPORT
    print("Training Complete. Exporting to ONNX for Jetson...")
    model.export(format='onnx', dynamic=True)
if __name__ == '__main__':
    train_segmentation_model()
```

Figure 2.7: Navigation Obstacle Detection Training Code

This code uses a segmentation model “yolov8s-seg.pt” as a base model to train on the hospital environment dataset with 28,761 images. The main parts of the code are:

- Model_name: loads the pre-trained YOLOv8 small segmentation model
- Datasets_yaml: the path to the Roboflow dataset YAML file containing train/val/test splits
- Training Parameters - Set to 50 epochs with batch size 4 to fit within 6 GB Vram for the GPU memory
- Early Stopping, so patience = 10 means that the training will stop if the validation loss doesn't improve for 10 consecutive epochs
- GPU Configuration used device=0 with 4 worker threads for data loading

- Export, automatically exports the best model to ONNX format after training completes so it can run smoother on the jetson more than a pytorch file ".pt".

2. Medication Detection Training Script:

This code trains a detection model "yolov8s.pt" on a medication dataset containing 360 images. The key differences from segmentation training are:

- Detection instead of Segmentation so the training uses yolov8s.pt instead of yolov8s-seg.pt
- More epochs because dataset is smaller and it can do 100 epochs much faster
- Larger batch Size since the dataset is less than the navigation so a batch of 8 is safe for the medication dataset size
- Higher patience at 15 epochs patience because detection training is more stable
- Detection task focuses on bounding box accuracy only, not pixel-level mask

```python
from ultralytics import YOLO


def train_medication_model():  1 usage

    model_name = 'yolov8s.pt'
    dataset_yaml = r"D:\05. Badr learning\04. University\Year 4 Sem 1\02. GDP\Brainstorming\AGV Camera\Datasets\Hospital_Meds_ZED.v1-hospital_meds_zed_01.yolov8\data.yaml"
    print(f"Loading {model_name} for Medication Task...")
    model = YOLO(model_name)

    #  TRAINING
    results = model.train(
        data=dataset_yaml,
        epochs=100,
        patience=15,  # Stop if it stops learning
        imgsz=640,
        batch=8,
        device=0,  # Use GPU
        workers=4,
        project='Hospital_AGV_Project',
        name='zed_meds_v1_detection',
        exist_ok=True,
        plots=True
    )

    # EXPORT
    print("Medication Training Complete. Exporting to ONNX...")
    model.export(format='onnx', dynamic=True)

if __name__ == '__main__':
    train_medication_model()
```

Figure 2.8: Medication Training Code

Both training codes generated:

- Confusion matrices
- Box precision-recall curves
- Training/validation loss curves
- Sample images for each task
- Training logs as .csv file
- Training code configuration saved as args.yaml
- Final ONNX models ready for deployment on Jetson

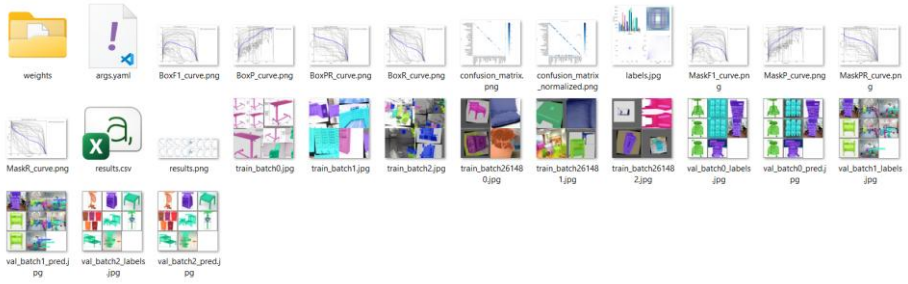


Figure 2.9: Trained models files

2.6.3 Testing GUIs on Laptop

Two Tkinter GUIs were created on the laptop to validate the trained models before Jetson deployment. These GUIs allow real-time testing with adjustable confidence thresholds and visualization of model predictions.

1. Obstacle Detection GUI

This GUI tests the segmentation model for detecting obstacles in hospital corridors. The main features are:

- Model Loading loads best.pt from the training directory
- Camera input captures video from the Zed 2i camera
- Inference runs YOLOv8 segmentation on each frame
- Confidence slider allows adjusting detection threshold from 0.1 to 1.0
- Mask visualization is used to show or hide segmentation masks
- Threading, so a video processing runs in separate thread to keep GUI responsive

2. Medication Detection GUI

This GUI tests the medication detection model with a "Sniper Mode" feature that simulates the AGV's focused verification behavior. Key features:

- High-resolution Input - Captures 1920x1080 from webcam for better medication recognition
- Sniper Mode Toggle - Simulates digital zoom by cropping center 640x640 region
- Dual View Modes - Shows both wide-angle normal view and focused sniper mode
- Background Darkening - Highlights the zoomed region by darkening surroundings
- Real-time Confidence Adjustment - Slider allows tuning detection sensitivity

2.6.4 Jetson Orin Nano Environment Setup

The Jetson Orin Nano is an embedded device with ARM-based CPU and NVIDIA Tegra GPU and I focus on AI application. Additionally, it runs a Linux OS.

Table 2.7: Jetson Orin Nano Hardware Specifications

Component	Specification
Device Name	NVIDIA Jetson Orin Nano
OS	Ubuntu 22.04.5 LTS (64-bit, Jammy Jellyfish)
Processor	ARMv8 CPU (8 cores)
GPU	NVIDIA Tegra Orin (nvgpu) integrated graphics
Memory	7.4 GB LPDDR5 RAM
Storage	500 GB NVMe SSD
CUDA Version	12.0 with v12.6.68 compiler toolchain
Python Version	3.10.12 (pre-installed with JetPack 6.0)
Jetson L4T Version	L4T Release 4.7 (Linux for Tegra)
Kernel	Linux 5.15.148-tegra

Table 2.8: Jetson Python Libraries

Package	Version
torch	2.8.0
torchvision	0.23.0
ultralytics	8.3.241
onnxruntime-gpu	1.23.0
opencv-python	4.12.0.88
numpy	1.26.4
pyzed	5.1
PyQt5	5.15.6

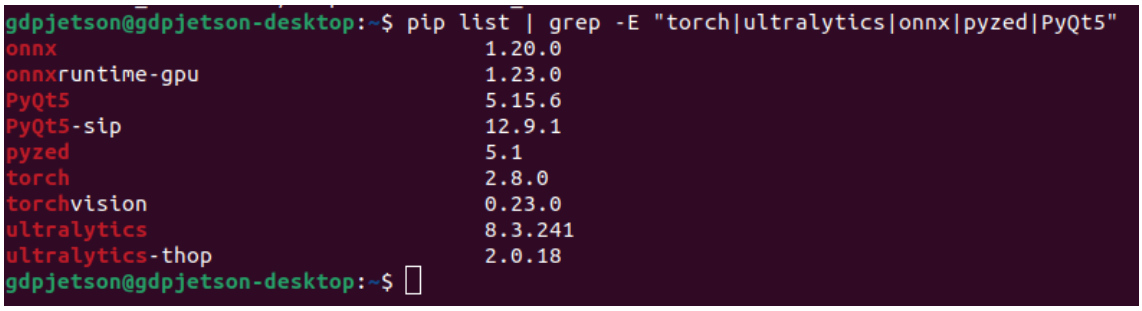


Figure 2.10: Jetson's Python Libraries

2.6.5 Jetson Deployment GUIs

The Jetson runs two PyQt5-based GUIs, each handling a different task, and both GUIs use ONNX models for fast GPU-accelerated inference on the Tegra Orin processor.

1. Obstacle Detection for Navigation GUI:

This GUI is the primary safety system that detects obstacles and measures distances using the ZED 2i depth sensor which controls AGV movement using a traffic light system (Red = Stop, Orange = Slow, Green = Go).

Key Functionality:

- Runs segmentation model on every frame at ~30 FPS for the obstacle detection task.
- Depth measurement extracts distance from depth map from the zed 2i camera at detected objects.
- Safety Decision Logic which compares minimum distance to thresholds (1.0m = critical, 2.0m = warning).
- Status display shows large colored banner with current status and closest obstacle name.
- LIDAR backup includes title mentioning LIDAR integration for redundancy.
- Obstacle logs maintain scrolling list of detected objects and distances.

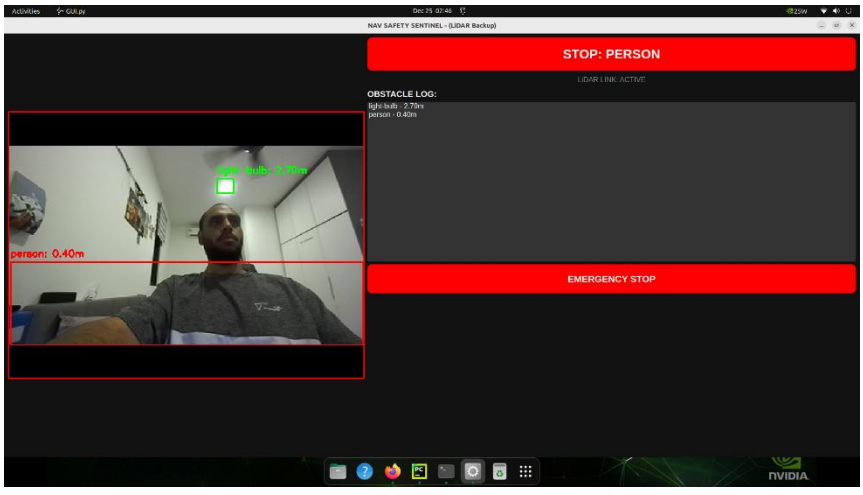


Figure 2.11: Obstacles Detection Models

2. Medication GUI:

This GUI runs the medication detection model to identify medicines being transported, it displays detected medicines in a list and shows confidence scores on the video feed.

Key Functionality:

- Medicine Detection runs detection model on ZED video stream
- The inventory logs maintain list of detected medicines without duplicates
- Confidence display shows detection confidence percentages
- Reset button to clear detected list and start fresh scan

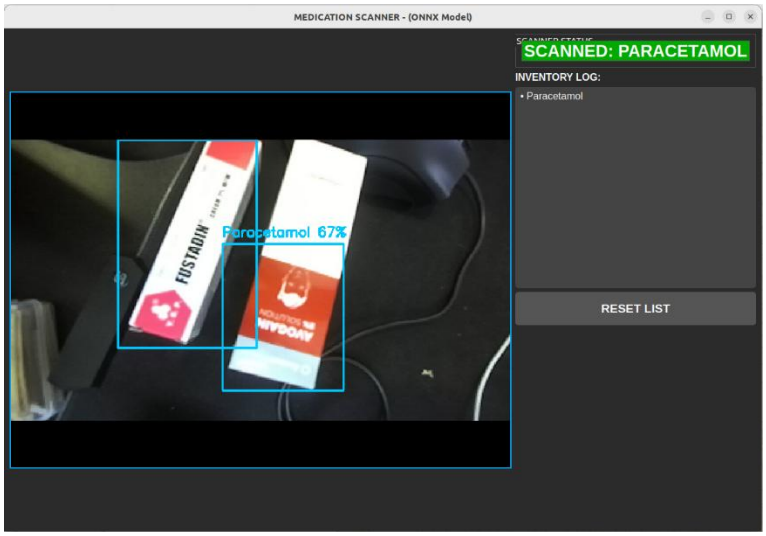


Figure 2.12: Meds GUI

2.7 Summary

In summary, this chapter described the development of the vision system for the obstacle detection and medication sortation for the AGV. Moreover, it justified the selection of the NVIDIA Jetson Orin Nano and ZED 2i camera to ensure performance stability. Furthermore, it explained the training of YOLOv8 models, and the logic used for safe obstacle avoidance. Finally, it detailed the implementation of the Safety Sentinel and Medication Scanner GUIs on the robot.

CHAPTER 3 AYSER MOHAMED (TP067377)

3.1 Introduction

Electrical and power management system acts as the base which all the autonomous navigation logic and the physical components of the AGV is built on. This chapter will go over design, component choice and implementation of the electrical subsystems needed to power all subsystems used for AGV to operate safely in hospitals. The system focuses on power distribution, motor drive method, motor control using an ESP32, and the safety mechanisms used to ensure that the AGV can operate under varying loads and conditions.

3.2 Literature review

3.2.1 Electrical Design of AGVs

The electrical design of an AGV is influential in assessing its reliability, safety, as well as scalability particularly in areas where safety is critical like hospitals. Modern AGV systems focused on a hierarchical electrical system with high-power circuits being isolated from low-power control, sensing, and computing systems, this minimizes the electromagnetic interference, makes diagnosing faults easier, and enhances the overall system. (Tejada et al, 2022) shows that AGVs designed with engineering based systems have the advantage in terms designing their modular power domains to have propulsion, control electronics, and sensing systems whose operation is controlled by independently regulated voltage rails. These types of architectures allows for easier upgrades and maintenance for the sub-systems without needing the entire system to be redesigned.

Bus-bar based power distribution systems are common in modern AGVs in order to handle high levels of current and still have low resistive losses. Additionally centralized bus bars with fused distribution blocks are simplier to wire than point-to-point wiring and are better thermal performers. (Vo Thu Ha et al, 2023) showed that centralized battery systems supplying distributed motor controllers with protected power buses are a suitable compromise between effectiveness, scalability, and security. This design philosophy is appropriate to hospital AGVs, where continuous and predictable electrical behaviour is needed.

3.2.2 Motor Control Systems:

The AGVs have motor control strategies that differ according to application requirements, controllers costs, and required smoothness of motion. The most common of these are the differential steering designs with simple mechanical characteristics based on autonomous control of left and right wheel velocities to generate both linear and rotational movement. One of the common ways of motor controls in AGV research is the use of PWM or DAC to generate analogue throttle signals as this provides a fine control over the velocity as well as a smooth acceleration curve. However, commercial low-cost motor controllers use discrete speed-level selection with a digital input, providing predefined speed modes which ease hardware implementation of the controller at the cost of motion resolution.

It was demonstrated that the AGVs that utilize discrete speed commands in combination with the higher-level motion planning and supervisory control could be relied on to perform reliable navigation (Zhang et al., 2019). Discrete controls are simpler than velocity commands, and simplicity makes the system smoother and software less complex. In the case of a hospital AGV with a relatively slow speed, this trade-off can be tolerated in the early phases of development. However, some studies suggest that a shift to continuous control in a later stage of development to minimize mechanical wear, ensure the safety of the supplies, and allow for better trajectory control.

Table 3.1 Summary of Literature Review

Author (Year)	Focus Area	Key Methodology	Key Outcomes	Application to Current Design
Zhang et al. (2019)	Control Architecture	Distributed processing (PC + MCU) to handle ROS navigation and low-level actuation separately.	Validated stable navigation and reduced CPU latency by offloading sensor data acquisition.	Dual-core ESP32 used to isolate real-time motor signals from the ROS 2 navigation stack.
Tejada et al. (2022)	Systems Engineering	Electrical isolation of high-power traction circuits from sensitive logic voltage rails.	Successful prototype testing with high reliability and immunity to electromagnetic interference (EMI).	**Power Isolation:** Implementation of separate 48V (Traction) and 5V/12V (Logic) buses linked via isolated buck converters.
Ha et al. (2023)	Motion Control	**Closed-Loop Feedback:** Implementation of PID algorithms using encoder data to correct trajectory errors.	Achieved precise path tracking and consistent velocity under variable payload conditions.	**Hall Sensor Feedback:** Utilization of motor Hall sensors to maintain torque on ramps and varying floor surfaces.

3.3 Investigation on Materials/Components Selection

3.3.1 Motor Selection

Table 3.2 Comparison of AGV Motors

Feature	**Geared DC Motor**	**Brushless Hub Motor**	**Stepper Motor**
Torque	High (due to gearbox)	Moderate (Direct Drive)	High (at low speed)
Noise Level	High (Gear whine)	Low (Silent)	Moderate (Whine)
Maintenance	Moderate (Brushes wear)	Low (Sealed, brushless)	Low
Efficiency	~60-70%	~85-90%	~60%
Complexity	Simple (PWM)	Moderate (Needs ESC)	Moderate (Drivers)

The AGV design will use a Xiaomi M365 brushless hub wheel motor, because it doesn't use a gearbox, thus removing the primary noise source in the AGV since it being able to operate quietly is important in for patients mental health, additionally it's hall sensors allows it to operate smoothly at low speeds which is essential in navigating the crowded corridors of the hospital.

3.3.2 Motor Controller Selection

Table 3.3 Comparison of Motor Controller Architectures

Feature	**RC ESC (Hobby)**	**Industrial PLC**	**Brainpower E-Bike Controller**
Interface	**PWM (1-2ms pulse)**	**Analog 0-10V / Fieldbus**	**Digital Logic / Hall**
Cost	**High**	**Very High**	**Low**
Current Handling	**Low-Med (Heat issues)**	**High**	**High (Robust sink)**
Sensor Support	**Often Sensorless**	**Encoder inputs**	**Hall Sensor Standard**
Max Current Rating	**30-50A**	**100A+**	**40A continuous**

From Table 3.3 it can be seen that to simplify implementation of differential steering in the AGV the Brainpower SL-KZSQ-X6 Dual Motor Controller was selected due to its dual control channels which reduces the wiring complexity. Additional its ability to receive digital signals which are more resistant to electrical noise allows it to operate even if the hall sensor fails increasing the reliability of the AGV. Lastly it being a cheap and reliable helps keep the cost needed to develop the AGV low.

3.3.3 Microcontroller Selection

Table 3.4 Comparison of Microcontrollers

Feature	Arduino Mega 2560	Raspberry Pi 4	ESP32 DevKit V1
Voltage Logic	5V	3.3V	3.3V
Processing Power	Low (16MHz)	Very High (1.5GHz)	High (240MHz Dual Core)
GPIO Pins	54	40	36 (PWM + ADC)
Connectivity	USB/Serial only	Ethernet/WiFi	Built-in WiFi/BT
OS Support	None (Bare metal)	Linux (ROS Master)	FreeRTOS / micro-ROS

As shown in Table 3.4 the Esp32 was chosen due to how it offers a balance between the I/O pins count and the processing speed. With its dual core architecture it is able to use one core to handle the safety loops and the other to manage communication. Moreover the built in Wifi and BT modules gives it more flexibility in how it communicates with the main processing unit and the other sensors.

3.4 Methodology

3.4.1 Power Distribution

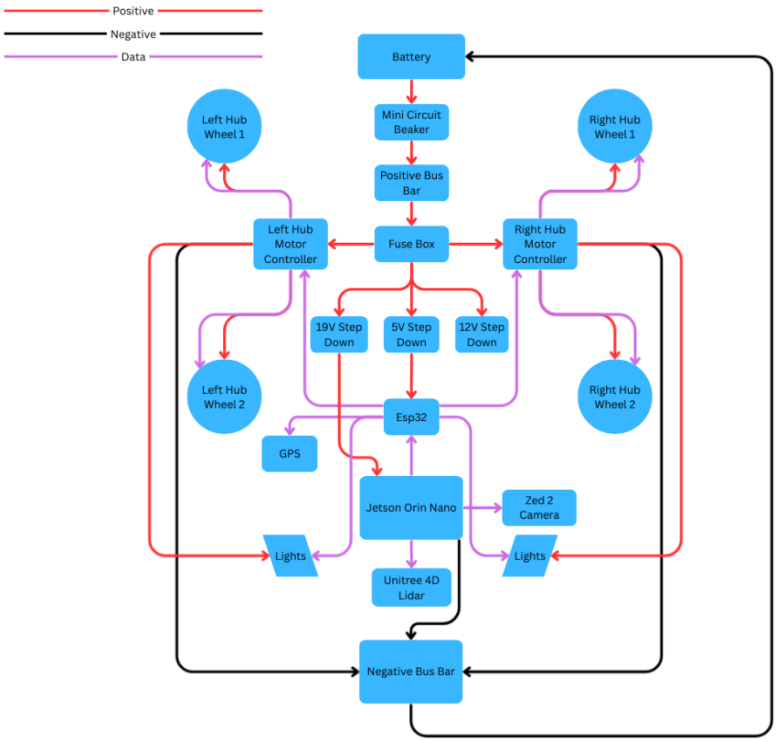


Figure 3.1 Power Distribution System Block Diagram

Figure 3.1 shows the diagram of the power distribution system. starting with the battery, used to power the system, is connected to a circuit breaker to cut the power flow in case of any fault in the battery or the system, then the power goes to the positive bus bar to distribute the power into the controllers and three step down converters. The controllers power the hub motors and the lights.

As for the converters, they step down the power into three voltages to supply the lower voltage components. A 5V step down that powers the Esp32, a 12V step down which powers the Zed camera and LiDAR, and a 19V step down which powers the main processing unit of the system the Jetson Orin Nano.

The Jetson is responsible for receiving the data form the camera and LiDAR and using it to chart the course needed to reach the task point, the course data is then sent to the Esp32 which then commands the controllers to change the speed of the tires to navigate based on the differential steering method. A negative bus bar acts as a stable ground for the system to close the electrical circuit

3.4.2 Motor Controls

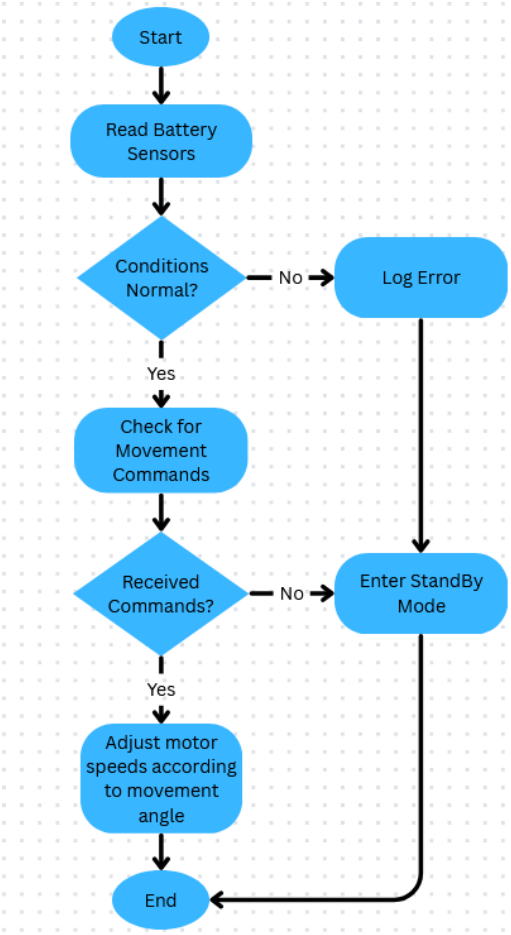


Figure 3.2 Motor Control System Flowchart

The designed system uses a sub-system motor and safety controls for the differential-drive AGV. The system will have an ESP32 acting as the secondary processing unit core, which will serve to connect the main processing unit to the motor controllers. As Figure 3.2 shows, the control process starts with the system being initialised, where the motor control pins and sensor inputs are being programmed to an appropriate safe default value. Furthermore, the ESP32 measures the voltage and temperature of the battery and the current running through the motors to keep the system within the safety limits. When an unsafe condition, like undervoltage or sustained overcurrent are detected, then all motor activity is instantly stopped by the controller and the error is logged.

Once the system is found to be safe, movement commands sent in by the main processor are decoded and converted into discrete speed and direction signals sent to both the left and right motor controllers. Differential steering is realized then through the control of the two motor controllers independently, which enable the AGV to proceed, reverse, or make turns.

3.5 Concept Design Derived from Fundamental Engineering Principles:

To ensure that the concept electrical design for the AGV is functional, it's important to ensure that the components selected can operate reliably and with safe limitations and for an acceptable duration, to achieve that calculation on key design aspects of the system such as resistive forces, power losses and operation time where done in this section, with their results then being used to modify the AGV design accordingly.

Table 3.5 Calculation Parameters Values and Assumptions

Parameter	Value	Source/ Assumption
Battery Voltage	48V	Selected Li-Ion Pack
Battery Capacity	100 Ah	Selected Li-Ion Pack
Battery Discharge	80%	Battery Lifespan Preservation
Total Mass	100 Kg	AGV + Equipment Load
Rolling Resistance Coefficient	0.015	Hospital Linoleum Flooring
Ramp Inclines	2 Degrees	Wheelchair ramp standard
Cruising Speed	1.5 m/s	Hospital safe speed
Motor Efficiency	87%	Avrage Hub Wheel Efficiency
Controller Efficiency	92%	BrainPower Controller Estimated Based on Testing
Step-Down Converter Efficiency	95%	Buck Converter Standard
AGV Width	0.6 m	Mechanical Design
Wheel Radius	0.108 m	Xiaomi M365 Size

3.5.1 Battery Runtime and Energy Calculations:

The total battery capacity could be calculated as:

$$Ecap = V \times C = 48\,V \times 100\,Ah = 4800\,Wh$$

The Mechanical Power Requirements could be calculated as:

The Rolling Resistance:

$$F_{rr} = m \times g \times C_{rr} = 100 \times 9.81 \times 0.015 = 14.7\,N$$

Where:

m is the total AGV mass

g is the gravitational acceleration

Crr is the floors resistance coefficient

The equation to calculate grade resistance of a ramp with a 2 degree incline is:

$$Fgrade = m \times g \times sin(2\circ) \approx 100 \times 9.81 \times 0.035 = 34.3\ N$$

Thus the total Resistive Force is:

$$Ftotal = Frr + Fgrade = 49\ N$$

And the mechanical power needed at the wheel could be found using the equation:

$$Pmech = Ftotal \times v = 49 \times 1.5 = 73.5\ W$$

This however is only and Ideal power without factoring the losses. By including the losses due to the motors, controllers and battery sequentially like:

$$Pmotor = \frac{73.5}{0.87} = 84.5\ W$$

$$P_{controller} = \frac{84.5}{0.92} = 91.8\ W$$

$$P_{battery} = \frac{91.8}{0.95} \approx 97\ W$$

And adding the loads from the Esp32 and other low voltage electronics (Current, Voltage, Temp Sensors, etc) together assumed to be:

$$Plogic \approx 5\ W$$

The the total Cruse power is found out to be

$$Pcruise \approx 102\ W$$

To account for the losses accumulated due to operations like acceleration, breaking and other idle losses a high operation factor or 4.5 is applied to get the average usage power to b:

$$Pavg = 102 \times 4.5 \approx 477\ W$$

By limiting the battery discharge to 80% to preserve it's life span the total usable energy capacity is:

$$Eusable = 4800 \times 0.8 = 3840\ Wh$$

Using this the Avrage operational time for the AGV could be calculated as:

$$T = \frac{E_{usable}}{P_{avg}} = \frac{3840}{477} \approx 8.0\ hours$$

That means that the AGV can operate continuously for an average of 8 hours, confirming that with this design the system can cover a full shift before it needs to recharge ensuring that it can be incorporated to a hospital setting without disrupting the workflow of the staff by shutting down during tasks.

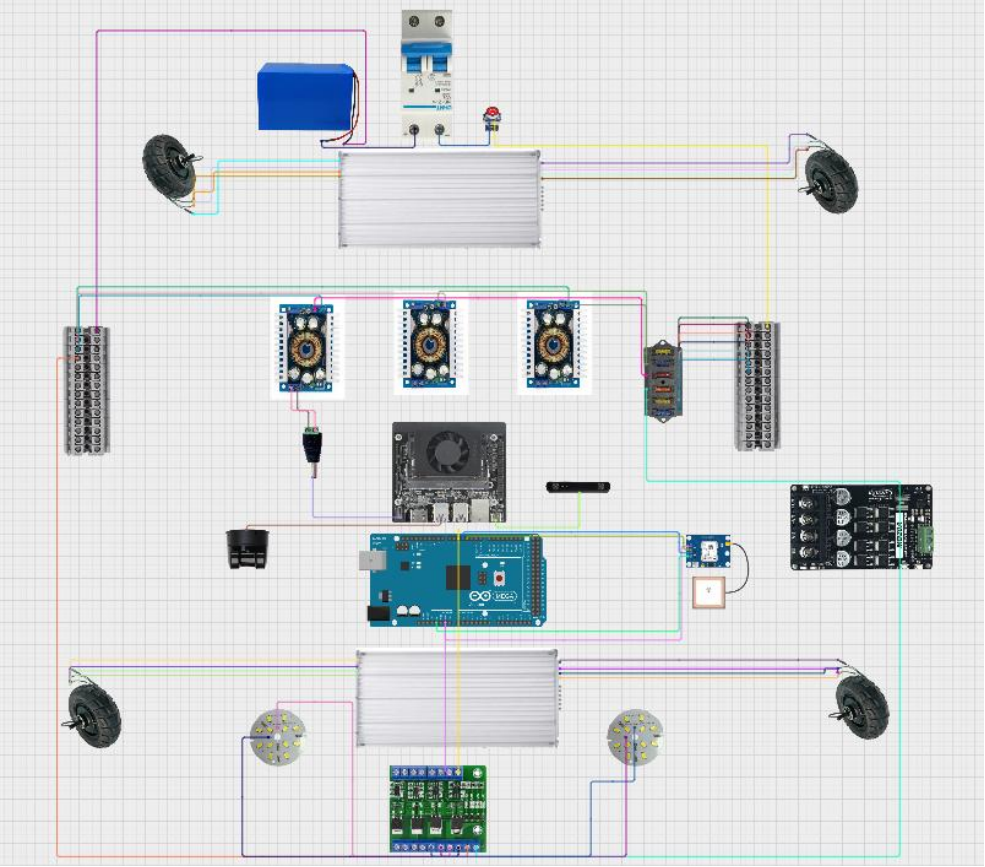


Figure 3.3 AGV Power Circuit Concept Design

Based on the results the AGV circuit was designed as shown in Figure 3.3 following the power distribution system. However, due to the high voltage of the battery relaying only on step down converters to protect the lower voltage components wouldn't be sufficient. As such additional safety measures were incorporated into the design like an E-stop button to allow the systems power to be shut down manually in case the circuit breaker fails, and a fuse box to protect the individual components from Overcurrent or current backflow; Ensuring that the system design is safe and reliable in accordance with fundamental engineering principles.

3.6 System Implementation

```ino
#include <Arduino.h>

/*CONFIGURATION */

const float TRACK_WIDTH = 0.6;     // Distance between left and right sides (m)
const float MAX_SPEED   = 1.5;     // Maximum linear speed (m/s)

enum SpeedLevel { STOP = 0, LOW_SPD, MED_SPD, HIGH_SPD };

/*Motor Side Definition*/
struct MotorSide {
  int pinLow;
  int pinMed;
  int pinHigh;
  int pinRev;
};

/*Left and Right Drive Sides */
MotorSide leftSide  = {26, 27, 14, 12};
MotorSide rightSide = {25, 33, 32, 13};

/*  LOW-LEVEL MOTOR CONTROL */

void setSideState(MotorSide &side, SpeedLevel speed, bool reverse) {

  // Deactivate all speed pins (active-LOW logic)
  digitalWrite(side.pinLow, HIGH);
  digitalWrite(side.pinMed, HIGH);
  digitalWrite(side.pinHigh, HIGH);

  // Set direction: LOW = forward, HIGH = reverse
  digitalWrite(side.pinRev, reverse ? HIGH : LOW);

  // Activate selected speed level
  switch (speed) {
```

Figure 3.4 Low-Level Control Code (1)

```ino
  // Activate selected speed level
  switch (speed) {
    case LOW_SPD:  digitalWrite(side.pinLow, LOW);  break;
    case MED_SPD:  digitalWrite(side.pinMed, LOW);  break;
    case HIGH_SPD: digitalWrite(side.pinHigh, LOW); break;
    case STOP:
    default: break;
  }
}

/*  SPEED QUANTIZATION */

SpeedLevel mapSpeedToLevel(float speedAbs) {
  float ratio = speedAbs / MAX_SPEED;

  if (ratio < 0.10) return STOP;
  if (ratio < 0.40) return LOW_SPD;
  if (ratio < 0.75) return MED_SPD;
  return HIGH_SPD;
}

/* DIFFERENTIAL STEERING  */

void driveDifferential(float v, float omega) {

  // Differential-drive kinematics
  float vRight = v + (omega * TRACK_WIDTH / 2.0);
  float vLeft  = v - (omega * TRACK_WIDTH / 2.0);

  // Limit speeds to hardware capability
  vRight = constrain(vRight, -MAX_SPEED, MAX_SPEED);
  vLeft  = constrain(vLeft,  -MAX_SPEED, MAX_SPEED);

  // Determine direction
```

Figure 3.5 Low-Level Control Code (2)

```
  // Determine direction
  bool rightReverse = (vRight < 0);
  bool leftReverse  = (vLeft  < 0);

  // Convert continuous speeds to discrete levels
  SpeedLevel rightSpeed = mapSpeedToLevel(abs(vRight));
  SpeedLevel leftSpeed  = mapSpeedToLevel(abs(vLeft));

  // Apply commands to each side
  setSideState(leftSide,  leftSpeed,  leftReverse);
  setSideState(rightSide, rightSpeed, rightReverse);
}

/*INITIALIZATION  */

void setup() {
  Serial.begin(115200);

  pinMode(leftSide.pinLow, OUTPUT);
  pinMode(leftSide.pinMed, OUTPUT);
  pinMode(leftSide.pinHigh, OUTPUT);
  pinMode(leftSide.pinRev, OUTPUT);

  pinMode(rightSide.pinLow, OUTPUT);
  pinMode(rightSide.pinMed, OUTPUT);
  pinMode(rightSide.pinHigh, OUTPUT);
  pinMode(rightSide.pinRev, OUTPUT);
}

/*  MAIN LOOP */

void loop() {

```

Figure 3.6 Low-Level Control Code (3)

The used Esp32 control code shown in Figures 3.4~3.6, achieves low-level differential steering controller through the grouping of the four hub motors into the left and right drive sides. At each drive side, there are two motors (front and rear) which are driven by one motor controller. This configuration is consistent with the designs of differential steering systems, under which the relative speed of the right and left sides of the vehicle, control the movement and not the direction of a single set of wheels.

On the hardware level, every drive side is connected to three active-LOW digital speed selection pins and one direction pin. The setsideState() function is used to guarantee safe operation, by disabling all pins before the desired speed mode is enabled. This prevents conflicting commands and the motor controller is not exposed to undefined electrical states. Direction control is also managed separately and each side has the option to move forward or reverse.

The basic logic of steering is carried out in driveDifferential() function. Having received a desired linear velocity and angular velocity, the ESP32 calculates the left and right side velocities that are needed by using the differential drive equations. These equations enable the AGV to do straight movements, slow turns and pivots through appropriate adjustment of the speed and direction of both sides.

Since the motor controllers only allow discrete speed modes, the calculated continuous velocities are translated to defined speed levels with the use of the mapSpeedToLevel() function. The direction is indicated by the sign while

the magnitude of each velocity indicates the level of speed being selected. This maintains the accuracy of the steering model without having to alter the digital control interface.

By implementing these functions we were able to control the motors speed and direction to decide the turning angle using only simple commands to the Esp32 as shown in Figure 3.7 these commands would be later sent form the main processing unit the Jetson Orin Nano automatically based on the navigated path.

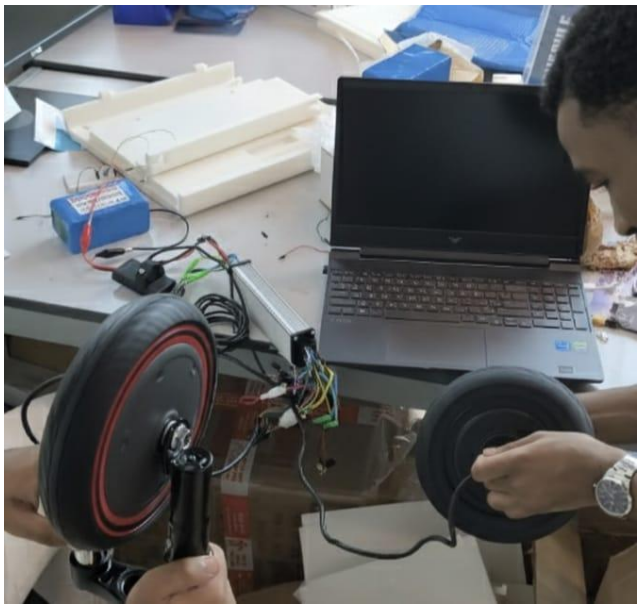


Figure 3.7 Motor Control Test Results

3.7 Summary

In summary this showed the design and implementation of the AGVs electrical and power distribution system. It also justified the component selection with comparison tables, explained the power distribution and motor control system designs, and validated the design using calculations and tested the control design to confirm that it can operate safely and reliably for sufficient durations when used in hospitals.

CHAPTER 4 ISAMELDIN AHMED TP069561

4.1 Introduction

This section focuses on the basic AGV design and its navigation. This section outlines 3D design schematics of the AGV frame, custom bracket designs to mount motor suspension, lidar and camera mounting brackets as well as a dedicated enclosure to store medical supplies and biohazards. Furthermore, ROS2- based navigation implemented used the Lidar will also be covered here.

4.2 Literature review

4.2.1 Autonomous Navigation in Hospitals

Autonomous Robots must be able to navigate spaces with non-moveable features such as furniture. In the context of this project, the AGV must be able to navigate hospital environments. These are spaces with high demand for safety moving obstacles and human interactions. Studies have highlighted that navigation systems for autonomous robots should include localization, mapping, preplanned path finding and especially collision avoidance. These systems often rely on sensors such as LiDAR and cameras which directly connect to a ROS navigation setup. Priority is emphasized here for safety and adaptability over smooth navigation, this is especially paramount for medical and biohazards carrying autonomous robots and vehicle (Bhat et al., 2024).

Researchers have introduced a structured protocol which comprises of four test phases with increasing levels of complexity for AGVs. A baseline of no obstacles must first be conducted. This involves a straight-line path navigation with no obstacles in the way of the AGV. Next static obstacles are added with one or two fixed obstacles positioned and different distances with each other. Mobile obstacles which are constantly moving are then added but the static obstacles are removed in this portion of the protocol. Finally, a combination of static and dynamic obstacles are implemented to mimic real world hospitals conditions an AGV may face. This approach covers both extremes an AGV may face from a completely clear path to a congested hospital (Bhat et al., 2024).

Figure 4.1: Simulated Hospital Environment
(Bhat et al., 2024).

Studies also concluded that navigation performance was critical to an AGVs success. They found that navigation performance decreases with an increase in path navigation and speed. This conclusion highlights the need for careful tuning of navigation parameters. Emphasis was also made on the high success rate of AGVs even in dynamic environments, however this conclusion was dedicated to the integration of reliable sensors such as LiDAR and cameras. They also emphasized the dimensions of the AGV relative to its environment also play a role in its effective navigation (Bhat et al., 2024).

4.2.2 Robotic Operating system for Autonomous Navigation

The design and implementation of omnidirectional AGVs often use the open-source robotic operating system also known as ROS. Research has often combined the use of a 2D laser or LiDAR rangefinder map with a 3D point-cloud map to achieve a map of real-time positioning. the use of oriented fasted and rotated brief feature slam to extract environmental features and positions is used here. This is done will performing simultaneous localization and mapping. The processes of global path finding is done here as well but instead using the A* Algorithm to find the quickest and optimal path. Furthermore, for local path

planning AGVs use the Time-elastic Band method. This employs the use of timed intervals between each segment to enable dynamic obstacle avoidance and smoother motion. This approach posed significant advantages when it came to performance as opposed to dynamic window approach. (Wu et al., 2023)

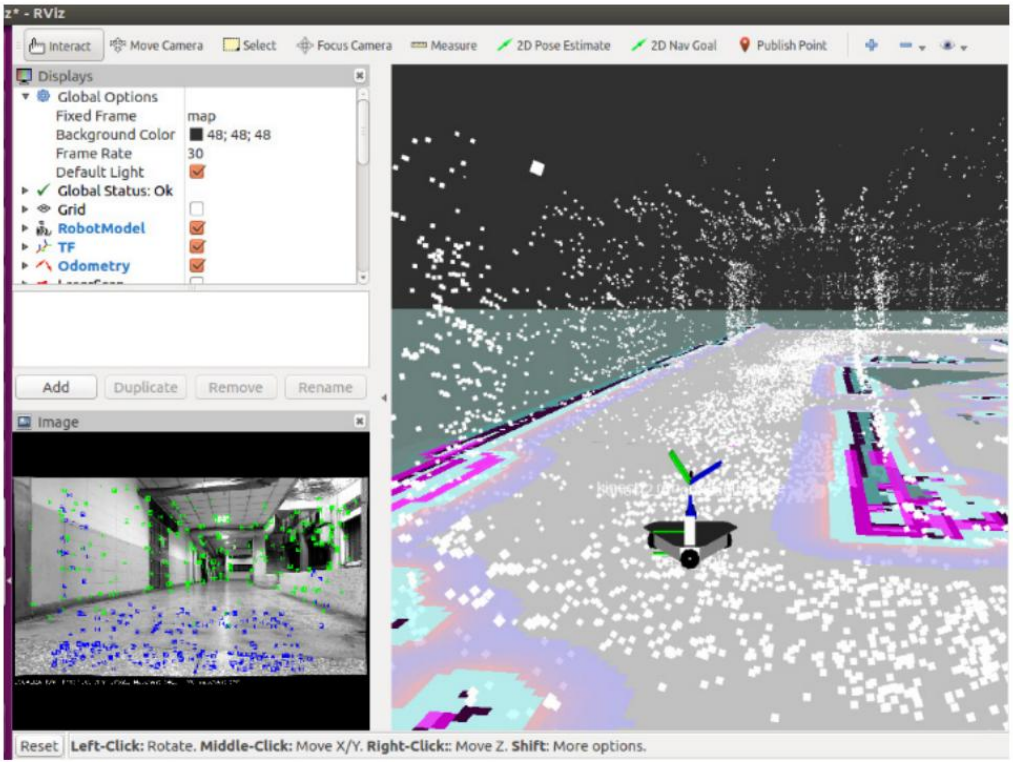


Figure 4.2: Point Cloud Map overlayed on Static Map

(Wu et al., 2023)

Figure 4.2 portrays green points which indicate ORB features from camera images while the blue points indicating the matched features used by the visual odometry to calculate the robot's position and its movement.

Robotic navigation is not without its limitations. Traditional AGV Pathfinding methods only account for short distance travel. To combat this, researchers have proposed a second path-planning strategy that integrates improvements in directed weight graph theory with ROS to consider both distance and steering costs. This is done to improve and increase efficiency with navigation. To achieve this this, environmental models with key nodes are identified first and optimized. These models are then converted into a directed weighted graph which represents movement costs which are detrimental to motion between the nodes. (Li & Liu, 2024)

Methods of improving navigation also involved the integration of algorithms into ROS. This method leverages the use of ROS' modular communication between nodes, real-time data processing and simulation

capabilities. Researchers' key implementation was the addition of the Floyd Algorithm. This is a global optimal path search which is also adapted to calculate the best paths on weighted graphs that includes distance and turning weights. This produces routes that reduce overall trip time and steering demands. Verifications for such methods were performed in a ROS-based Gazebo simulation, and the results were compared to traditional A* methods. It found that although A* finds a shorter path, it detriments steering and overall time costs as compared to improved weighted graph methods (Li & Liu, 2024)

Table 4.1: Summary of Literature Review

No.	Year of publication	Authors	Methodology	Outcomes / Advantages
1	2024	Bhat, A., Vallicrosa, G., Sanz, D. M., & Torroella, F.	LiDAR-based SLAM, ROS navigation, structured benchmarking test protocol with static and dynamic obstacles	Demonstrated navigation performance under hospital-like conditions. Emphasized robust localization, obstacle avoidance and safety margins for indoor environments
2	2023	Pan-Long Wu, Jyun-Jhen Li and Jinsiang Shaw	ORB-SLAM with LiDAR and camera sensors, A* global planner, TEB local planner within ROS	Improved localization accuracy. Smooth and effective obstacle avoidance. Demonstrated feasibility of integrating SLAM and ROS for AGVs
3	2024	Yinping Li and Li Liu	Directed weighted graph path planning considering distance and turning costs, Floyd algorithm as well as ROS and Gazebo simulations	Optimized AGVs path considering turning and distance. Emphasized smoother and more practical motion as well improved path planning efficiency with ROS integration

4.3 Investigation on Material and Components Selection

To design an AGVs navigation system and chassis frame, careful consideration must be taken on material and component selections. This is done to ensure optimal performance, structural integrating and computability with autonomous navigation requirements.

4.3.1 LiDAR

Selection of a LiDAR module is paramount to the systems success. The AGVs navigation hinges on its reliable movement within indoor and outdoor hospital environments.

Table 4.2: Comparison of LiDAR Types

LiDAR Type	Measurements	Key Capabilities	Typical Uses	Advantages and Limitations
2D LiDAR	Distance measurements on a single horizontal plane	Scans in a flat 360° circle around the robot	Basic indoor navigation, simple obstacle avoidance and floor-level mapping	Lower cost and simple to process cannot detect obstacles above or below the scanning plane
3D LiDAR	X, Y, Z spatial coordinates, generating full 3D point clouds	Captures multiple lasers at different angles to map vertical and horizontal structure	Complex environment mapping in autonomous vehicles and detailed SLAM	High-resolution environmental perception; detects shapes and obstacles at different heights; higher cost and data volume than 2D
4D LiDAR	X, Y, Z spatial coordinates and velocity	Adds direct measurement of object movement	High-dynamic environments, autonomous driving and	Velocity information for moving objects and better detection range more

		using velocity data	robotics tracking moving objects	expensive and complex to integrate

Table 4.2 discusses various lidar types. It highlights 2D as being the cheapest option and easiest to implement but has its limitations as it cannot detect obstacles below and above and can only see in front and behind its scanning plane. 3D LiDAR is the next step above. It captures a full 3D dimensional spatial environment. It allows for the possibility to navigate around obstacles as it can detect them at varying heights. Finally, 4D LiDAR combines 3D with the inclusion of motion and velocity information to allow the AGV to track dynamic obstacles. This will allow the AGV to react to changes in its surroundings and avoid collisions.

For this project, the use of a 4D LiDAR has been selected. It provides comprehensive perception of surroundings with the added benefit of motion data. The Unitree 4D LiDAR was selected for its precise 360° scanning capabilities. It enables high detection range and the added benefit of ROS compatibility.

4.3.2 Control System

The selection of the AGVs baseline of thinking and navigation was given to the open-source robotic operating system, ROS. It was selected due to its compatibility with sensors and control algorithms for navigation. It supports modular node-based sensor fusion, SLAM and path planning. With it being open source, it has extensive community support and libraries for LiDAR, cameras and motion control. It also enables the use of simulations and real-world testing using Gazebo and RViz.

Its implementation as opposed to other proprietary control system is emphasized by their lack of flexibility. Proprietary control systems or PLC-based solutions often offer stability but lack any flexibility in terms of rapid algorithm integration and sensor integration. ROS overcomes this while still having the capabilities for obstacle avoidance and navigation stacks which are essential for dynamic environment AGV operations.

4.4 Methodology

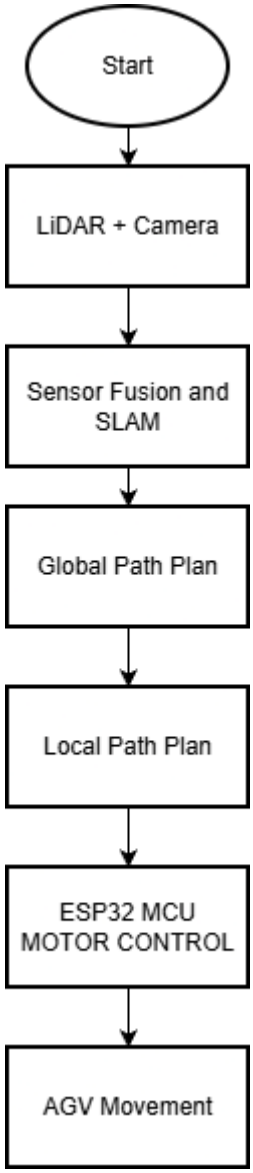


Figure 4.3: ROS algorithm Flowchart

Figure 4.3 conveys the AGVs navigation flow logic as it is placed in a hospital environment. The LiDAR and camera will be the first basis of input into the system. They gather information data of the environment and everything around the AGV. Sensor Fusion and SLAM will use this information to generate maps and estimate the position of the AGV and movements. The A* Algorithm will be the global path planning of the AGV to determine the optimal route for it to take. Local path planning is then implemented to ensure smooth navigation why the AGV is moving and obstacle avoidance. This control information for movement will then be sent to ESP32 microcontroller to then drive the AGV motors for movement.

4.4.1 Global and Local Path Planning

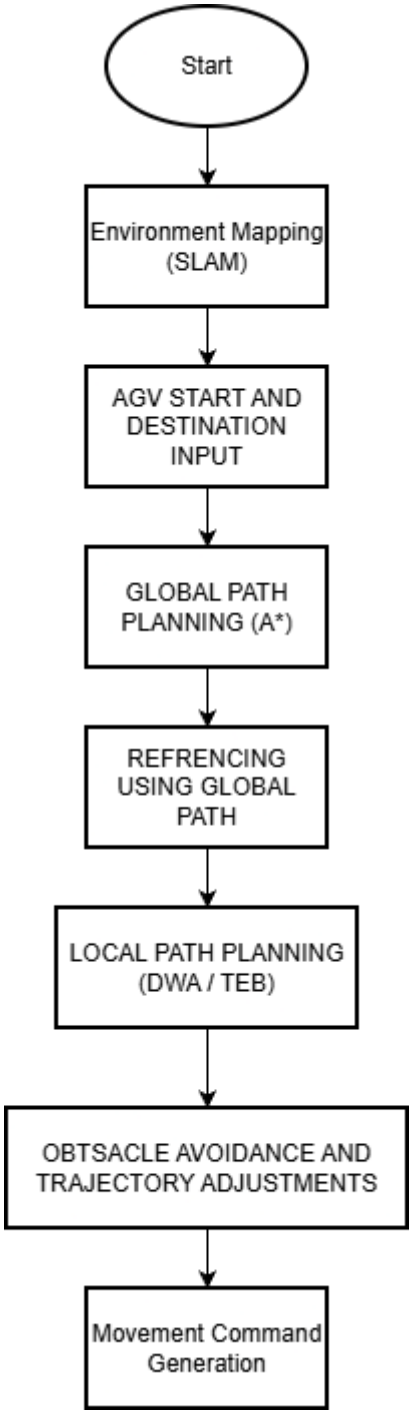


Figure 4.5: Global and local path planning flow chart

Figure 4.5 Ooutlines the general algorithm for navigation of the AGV. SLAM is used to generate a static map of the environment while the global planner will determine the optimal route using the A* Algorithm. The local planner will adapt and adjust the AGVs motion in real time.

The local planner is implemented to enable real-time trajectory adjustments and obstacle avoidance. It carries this out while doing the global path. A local planner such as the Dynamic window approach is used within the ROS Navigation Framework. The local planner continuously processes sensor information from the LiDAR and camera to avoid any obstacles the AGV may encounter. It adjusts the AGVs velocity and movement commands according to any obstacles prevalent.

4.5 Concept Design Derived from Fundamental Engineering Principles

4.5.1 AGV Chassis Design

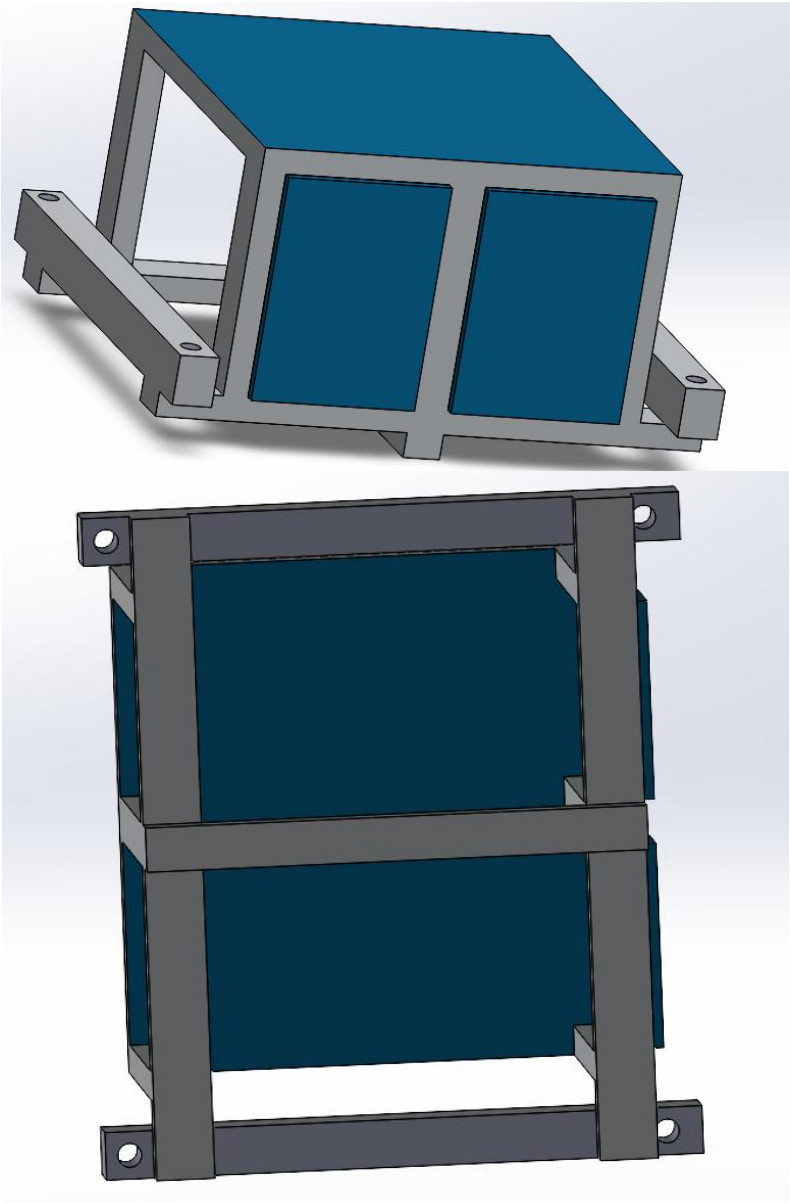
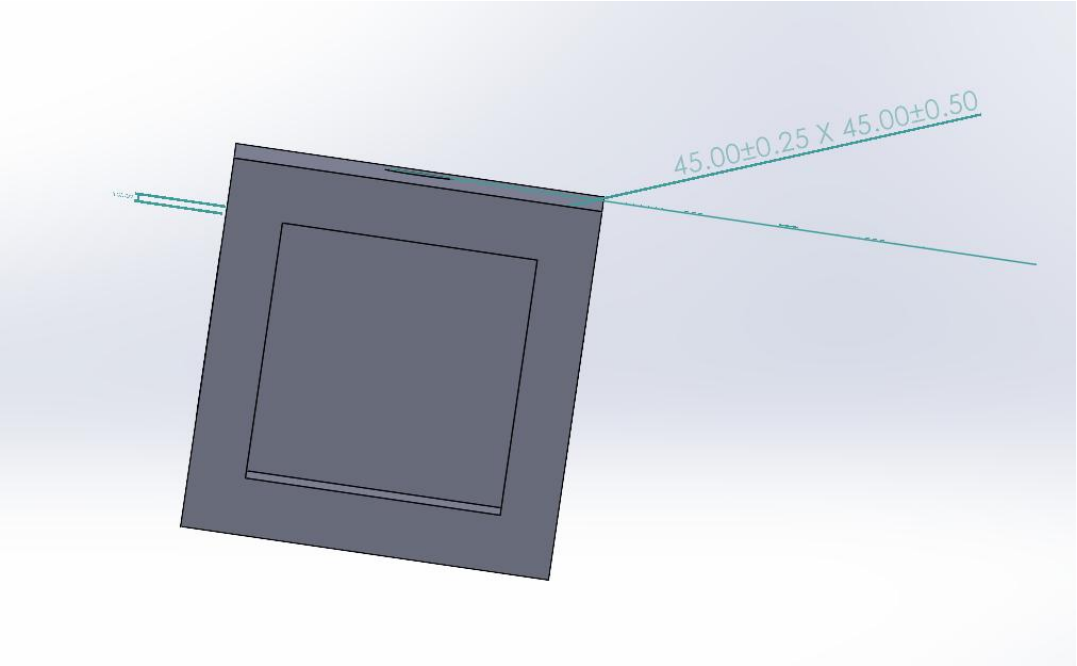


Figure 4.5: AGV Chassis

The AGV chassis was designed in SolidWorks to get a general blueprint of its design. It consists of 120cm length and 100cm in width. It provides a stable platform for transporting medical equipment and biohazards equipment in hospital environments. The dimensions of the AGV allow it to be fully capable of carrying these supplies.

The frame of the AGV will be made using aluminium profiles. A mixture of 45x45 and 40x80 aluminium profiles will be used here for their structural rigidity and modularity.

4.5.2 Suspension Frame Attachment



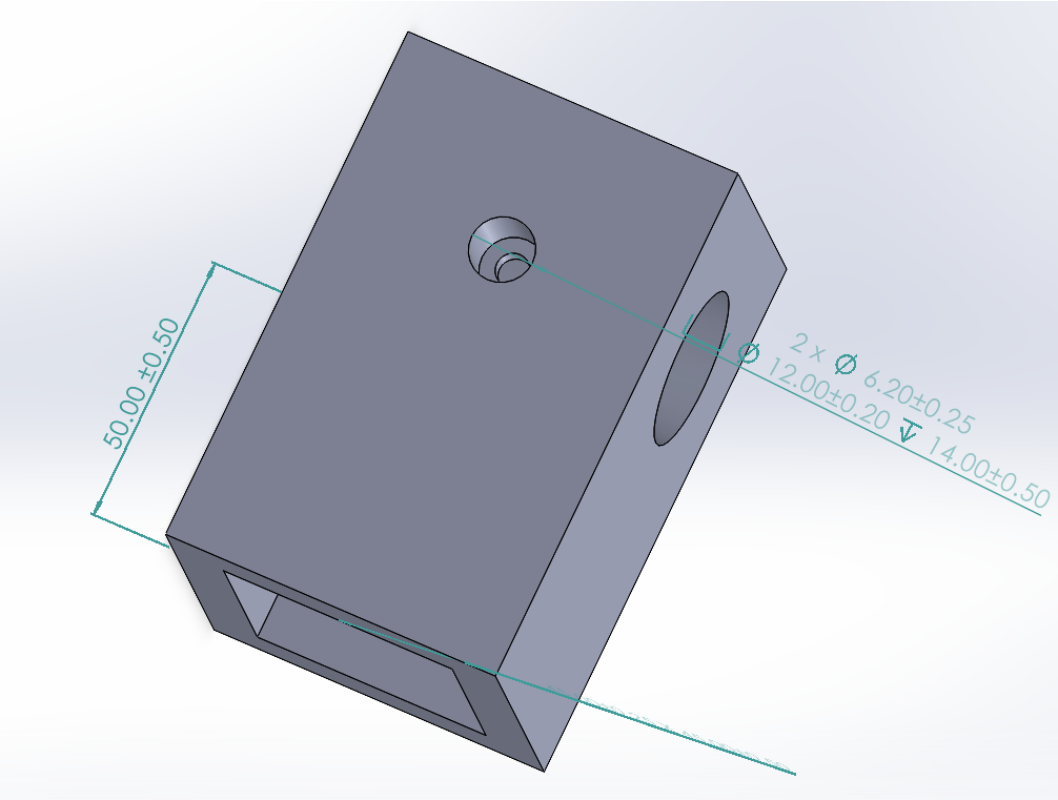


Figure 4.6: 45x45 Aluminium Profile Suspension bracket

In order to attach the suspension system to the chassis, a custom bracket that would attach to the end of a 45 x 45mm aluminium extrusion frame. The bracket slot was made to the exact size of 45mm x 45mm. the aluminium will slide into the bracket with 50mm of it going inside the bracket.

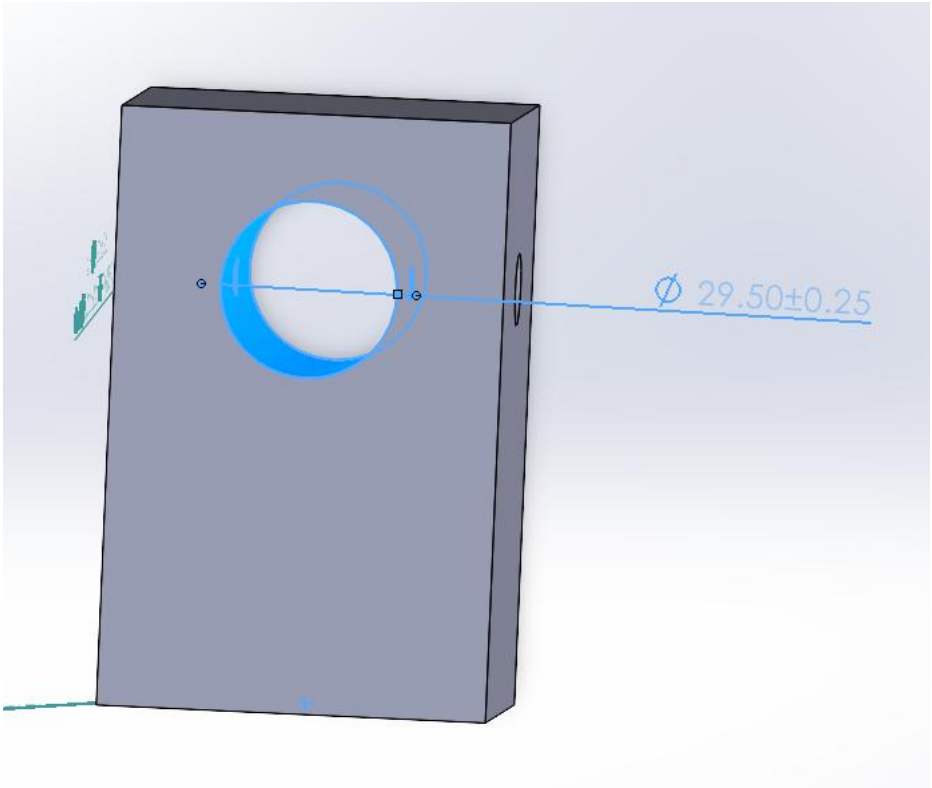


Figure 4.7: Suspension hole for Bracket

Next, a 29.5mm hole for the suspension strut to slide into was cut. Holes to screw the suspension into were also added perpendicular to the main strut hole. M6 screws on each side were used here for structural rigidity.

4.5.3 Unitree 4D LiDAR Bracket

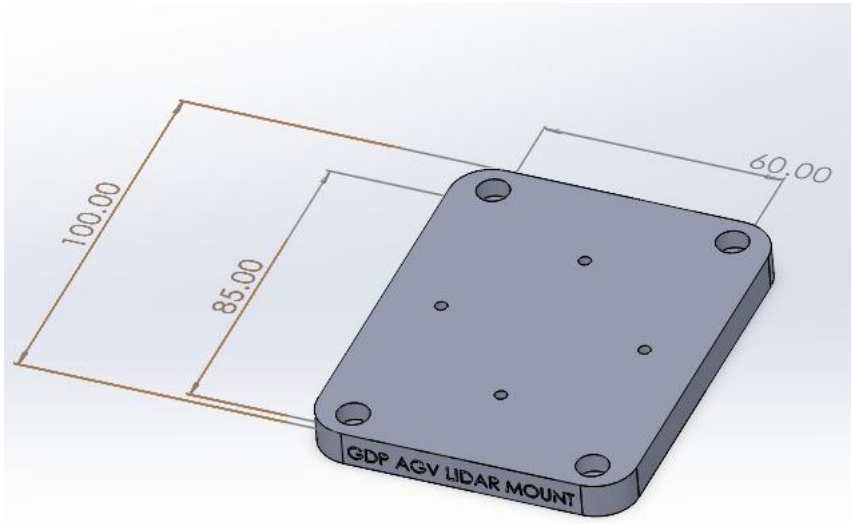
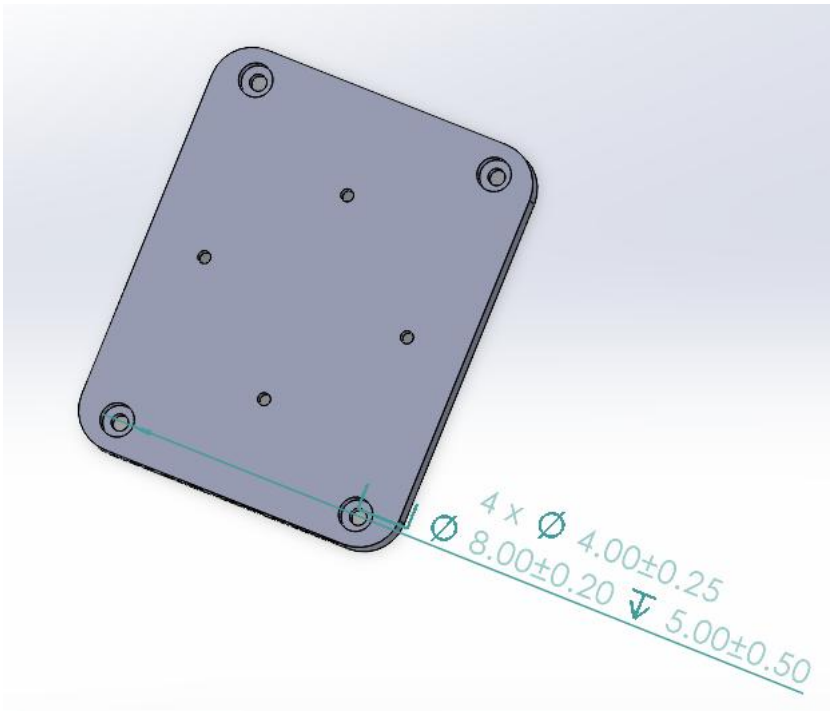


Figure 4.8: LiDAR Mount

The lidar of the AGV will be its main way of navigation around hospital environment. Thus, a mount for it to be placed on the AGV for it have a vantage point while not affecting the cameras' ability to function. A camera base of 100mm x 60mm was first extruded as shown in **Figure 4.8.**



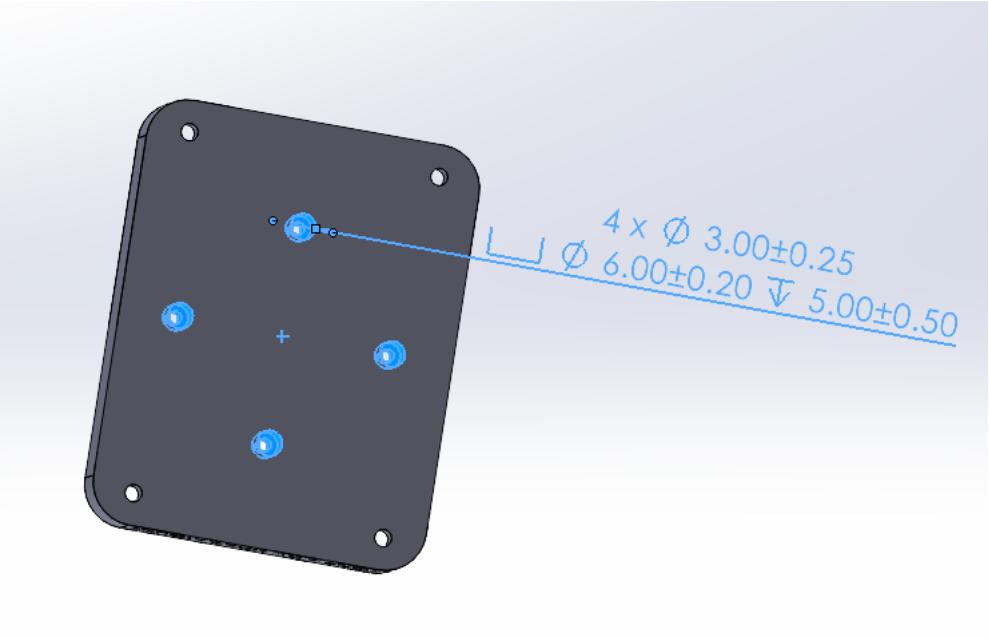


Figure 4.9: LiDAR Mount Screw Holes

Adjacent screw mounts for the lidar were added as shown in **figure 4.9.** the unitree LiDAR uses 4, M3 screws to hold onto brackets and mounts. Furthermore, an outer set of screw holes were added to the mount with M4 screw holes to be able to attach to the chassis.

4.6 System Implementation

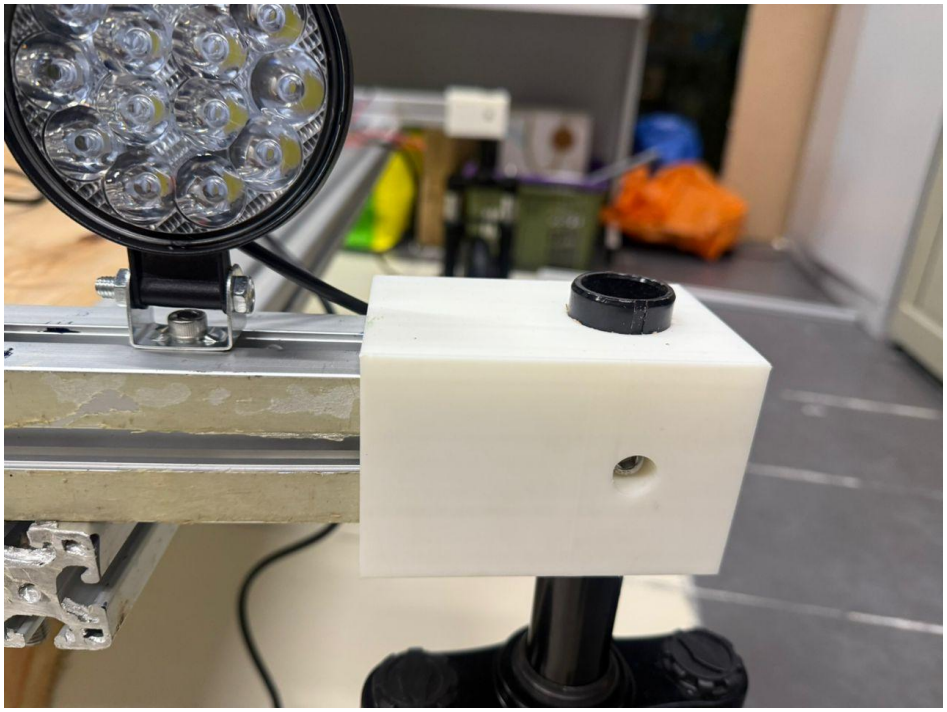


Figure 4.10 : Suspension Bracket

Figure 4.10 portrays the suspension bracket. It was fabricated using PLA+ through 3D printing and mounted onto the 45 X 45mm aluminium extrusion of the AGV. The printed bracket securely holds the suspension in place with no wobbles of creaks ensuring proper alignment and load transfer during motion of

the AGV. The use of 3D printing allowed rapid prototyping and design iterations to the bracket.

Figure 4.11: LiDAR Mounted to Bracket

The custom LiDAR bracket was also printed use PLA+ as shown in Figure 4.11. the bracket was attached using the provided M3 screw the Unitree 4D LiDAR ships with. The bracket positions the lidar at an optimal height to ensure stable and accurate environmental scanning. Structural rigidity was prioritized in the design to minimize vibrations that could affect sensor accuracy during AGV motion

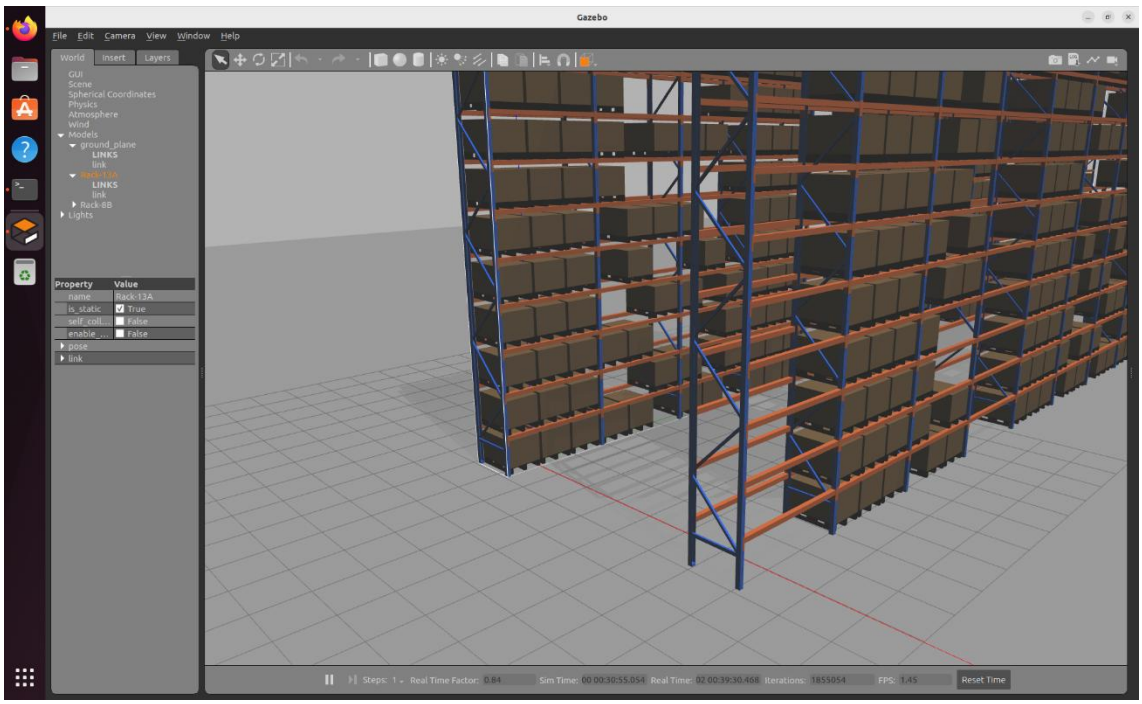


Figure 4.12: Gazebo Simulation with Static Obstacles

A Gazebo Simulation environment was run and created to evaluate the AGVs navigational capabilities in the presence of non-moving objects. The simulated environment represents an indoor setting with fixed racks. The AGV was deployed in the simulation with LiDAR and a camera configured to match the physical setup. Global path planning was carried out using the A* Algorithm and local path panning to avoid obstacles. The simulation provided a controlled environment for testing the real-world deployment of the AGV. However real-world deployment introduces uncertainties such as sensor noise and calibration tweaks.

Figure 4.12: Chassis Frame

The 3D design of the AGV was then fabricated to see the overall concept design. The overall frame dimensions of 120cm x 100cm, providing a stable platform for all mechanical and electronic components. Constructed using a mixture of 4080 and 4545 aluminium extrudes the frame design allowed for modular integration of components

4.7 Summary

In this section, several key components essential to the AGVs motion and general structure were designed and fabricated. Safe transport and autonomous navigation were key to this project's success. ROS was implemented using the LiDAR to enable autonomous motion while still enabling for input from the camera. The chassis frame provides a stable foundation for the system while 3D-printed wheel suspension brackets connect the suspension to the chassis. SLAM, Global and local path planning were integrated into ROS to allow for autonomous navigation and provided a steering instruction set. These instructions were later passed to the ESP32 for motor control. Together these components create a modular and reliable AGV platform capable of safely operating in a variety of environments for safe and reliable transport of medical supplies and biohazards.

CHAPTER 5 Rayan Al Yaqoubi TP063236 Individual Part

5.1 Introduction

In contemporary intelligent healthcare spaces, there is a strong necessity for secure access to operate AGVs. One excellent solution is card locking RFID with wireless control and real-time dashboard monitoring. RFID operates contactless card IDs that rely on individual personnel to enter sensitive compartments such as medication or biohazard units. This minimizes risks caused by lost keys and human oversight. For the wireless remote control, it can be operated freely when emergency or maintenance conditioning without interruption AGV operation. On-the-fly dashboard monitoring shows who has gained access, door status and AGV running for better traceability and visibility system-wide.

5.2 Literature Review

5.2.1 RFID Technology and Its Applications

Radio Frequency Identification (RFID) is an automatic identification technology; it uses a wireless communication method for transferring data from a tagged item to its reader devices without requiring human involvement. Profetto, Gherardelli and Iadanza (2022) performed a scoping review of RFID applications in health from 2017 to 2022 that emphasized implemented systems and functional prototypes. With a methodology inspired by PRISMA, we studied how RFID is used in practice for healthcare purposes and not just theoretical scenarios.

The article emphasizes the capability of RFID in real time data acquisition and automatic identification. Typically, an RFID system includes tags (transponders), readers and the backend processing equipment, in which each tag replies to a radio signal from a reader with its own unique identifier. This non-contact characteristic rendered RFID perfect for hospital applications, e.g., asset management, staff identification, and access control.

As compared to barcode systems, RFID has the advantages of non-line-of-sight operation, multiple tag reading at the same time, higher speed and more accurate data collection. The study also points up obstacles such as data security, privacy considerations, electromagnetic interference and deployment expense. These results are especially important for security designs of healthcare access control systems using RFID.

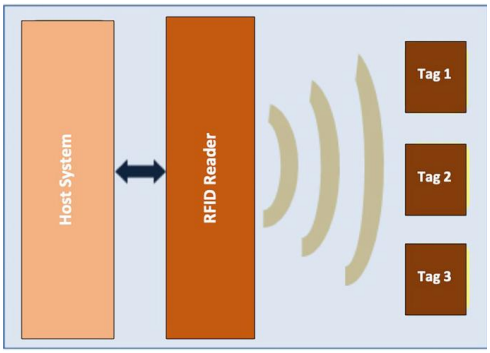


Figure 5.1: Basic Architecture of a Radio Frequency Identification (RFID) System

5.2.2 IoT Dashboards and Real-Time Monitoring in Smart Healthcare Systems

Abdulmalek et al. (2022) offer an extensive review on IoT-enabled intelligent healthcare monitoring systems, comfortable to the patient needs and evaluating our current research scenario of achieving real-time efficiency for personalized care, patient safety in a hospital environment of today. The work details how IoT-supportive architectures combine sensors, wireless communication links, processing nodes and cloud-based dashboards, which have led to continuous monitoring of certain critical parameters. Dashboards serve as centralized human-machine interfaces inside hospitals, permitting health workers to monitor the system state and take actions when operational or clinical events occur.

In the following review, IoT dashboards are underlined as control middleware of smart healthcare systems which plays a central role in integrating real-time data collected from different locations by channels like sensors, RFID readers, and automated equipment. This capability is especially important among speculative ones (e.g. AGVs), whose access control can propagate synchronously between the navigation and task execution monitoring. Real-time visualization and event logging also contribute to reducing human error, increasing traceability and ensuring regulatory compliance. However, the issues of data security, interoperability and scalability need to be addressed. In conclusion, the study highlights the importance of dashboards to assist secure, transparent and efficient health care automation systems.

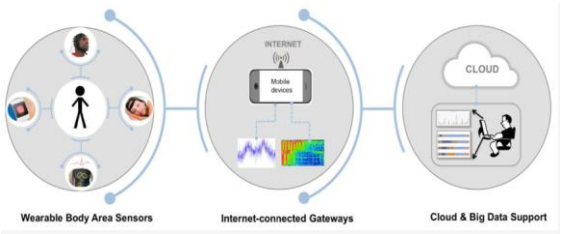


Figure 5.2: IoT-Based Healthcare Monitoring System

5.2.3 Web-Based Real-Time Teleoperation Control Framework

Pico et al. (2025) introduced a real-time teleoperation system designed in the form of a web application, which can operate robotic platforms remotely using networked interfaces and standard communication protocols like ROS1 and ROS2. The system enables human–robot interaction through a web-based graphical user interface, where the operators can issue direct control inputs and monitor robot responses in real time. The platform is built around web technologies permitting it to be device-agnostic and accessed from any device without special software.

The real-time localizing, sensor feedback and command sending are combined to guarantee responsive and reliable control of the robot. A permanent data exchange between the robot and the web interface also improves situational awareness in contexts where comprehensive autonomous navigation is not possible or would be hazardous. The performance of Internet-based teleoperation demonstrates the feasibility of autonomous systems for flexible, safe and reliable application by providing human-in-the-loop intervention in case of navigation failure or unforeseen event. In total, this work adds credence to web-enabled teleoperation as a technique for versatile and scalable robotic control in changing environments.

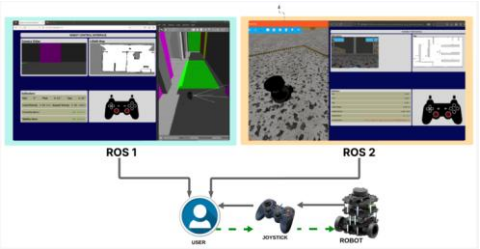


Figure 5.3: Web interfaces for a teleoperation system

5.3 Investigation on Materials/ Components Selection

Figure 5.4: Arduino IDE

The Arduino IDE was used for the development, testing and debugging of hardware system -BT-model and integration with the Blynk IOT platform. Preliminary test programs have confirmed that the RFUID can read UID via serial monitor. On validation, the IDE supported live data communication with Blynk dashboard. Log analysis helped debugging of RFID data, connections and command execution during serial monitoring.

Figure 5.5: Blynk Platform

A monitoring and control UI of real time AGV that utilize Blynk was implemented. Battery gauge, movement progression, type of task and state of the medicine and biohazard doors are all displayed on the screen. It also includes emergency stop and return home commands to ensure safe, hands-off monitoring, ongoing system condition updates and uninterrupted AGV operation without the risk of physical contact.

Figure 5.6: RFID RC522

Sensor and communication devices were selected to provide safe integration of the AGV. For contactless access control RFID was selected, with the mobile tag's unique ID also being checked by the microcontroller. It controls independent medication and biohazard compartments restricting access to

authorized users only. Once authenticated, the door automatically opens and the "Ready to Compare" dashboard log immediately updates.

Figure 5.7: Arduino Uno

The AGV access control system is designed with the Arduino Uno because of its simplicity, reliability and it can be interfaced easily with RFID systems. Powered by an ATmega328P, it has enough power and the I/O you need to play with homemade NFC stocks (including SPI for RFID communication). RFID was selected due to its contactless operation and secure manner in the sense that only those tags whose individual identity matches an authorized list succeeded. On a successful authentication, system actions and dashboard are updated to ensure safe and monitored execution.

Figure 5.8: FlySky FS-i6 Transmitter and Receiver

The wireless manual control for AGV was realized by the FlySky FS-i6 6-channel drone remote controller and receiver. It will be 2.4 GHz and the system provides stable low-latency communication. The control signals are transformed into PWM outputs for controlling the motors and each channel is assigned a function. A self-fail-safe function ceases AGV operation when signal is lost.

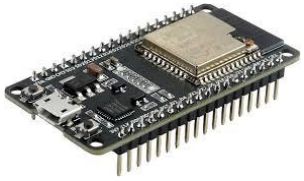


Figure 5.9: ESP32

The ESP32 chip was selected as the central controller for the AGV system because of its robust computing power and inclusion of Wi-Fi. It is dual core, enables both sensor processing and network communication at the same time. The ESP32 pulls data from the RFID reader and handles system logic, and sends

live updates to the Blynk dashboard. It is also controlled by AGV operation and security commands, so it can be applied to an IoT-based AGV application.

5.4 Methodology

It is equipped with accessibility, real time monitoring and remote operation of the AGV through security and dashboard system. This method utilizes an RFID authentication scheme and a web-based dashboard to make sure that only authorized personnel can enter the system to control it as well as keeping looking over AGV operation constantly.

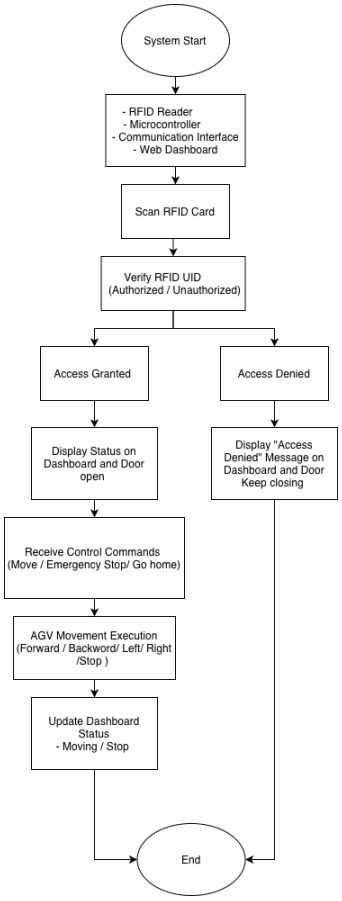


Figure 5.10: Control Flow Diagram of the Secure AGV System

At first, the RFID module, microcontroller and wireless communication interface are powered on. Pre-Registered RFID card UIDs are stored in the system. When an RFID card is swiped, the UID of the card will be read by the RFID reader and transmitted to microcontroller for checking against the authorized list. Access is allowed, and control functions are activated, if a valid match has been found; otherwise, access remains denied and the system is locked.

Once authenticated, the AGV connects to a Web-based dashboard which is used as a main control and monitoring interface. The dashboard makes it possible

to remotely control (e.g., movement commands or emergency stop) the robot while receiving real-time feedback about system health, operational status, and security. Safety measures guarantee that AGV will then assume a safe state when a communication failure or wrong command receival take place, which makes sure the system work safely and reliably.

5.5 Concept Design Derived from Fundamental Engineering Principles

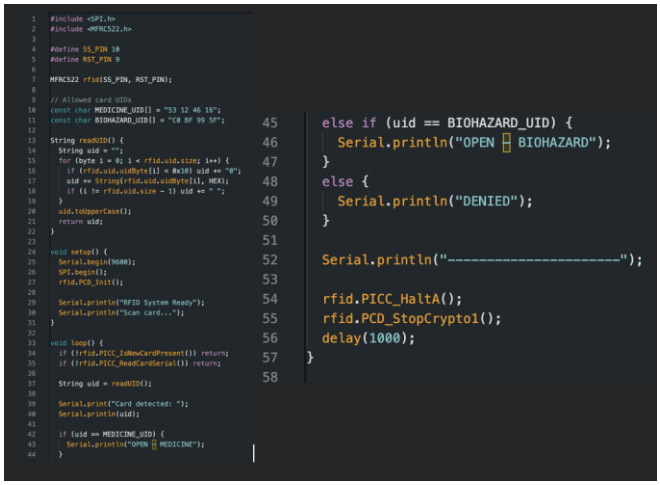
The AGV system proposed is fundamentally engineered to obtain reliable sensing, control and real time monitoring. With hardware-based inputs, built-in processing and software-driven decision making, the system provides reliable navigation, safe access control and constant operational awareness. The proposed framework based on RFID authentication, sensor monitoring, wireless transfer and display of the dashboards ensures a structured and reliable control for healthcare applications.

The system is designed as an embedded system and comprises a microcontroller, the ESP32 in this case serves as the central processing unit. It receives input from several ID sources: RFID readers, load cells, voltage sensors and IR receiving devices. These inputs are operated upon with predefined conditions to yield system states like whether the access is allowed, if tray load exists, battery status and motion control etc. This is where both digital and analog signal processing comes into the picture which helps in good lesion to sensor data interpretation followed by robust decision.

Control logic is written in both conditional and event-based programming. With authorized RFID authentication, compartment entry and AGV operation are possible as such the case that load and battery levels can be continually monitored for safety operating ranges. The AGV is wirelessly linked to a real-time dashboard which facilitates monitoring of the status of movement, clearance permissions and system condition from a distance. Interaction between human and machine is facilitated by the visual indications, warn-indications, and manually override functions like an emergency stop.

In general, the system fuses electronics, embedded control and communication reasons to realize the secure access control systems, real-time monitoring systems and reliable AGV running for assistive/healthcare application.

5.6 System Implementation

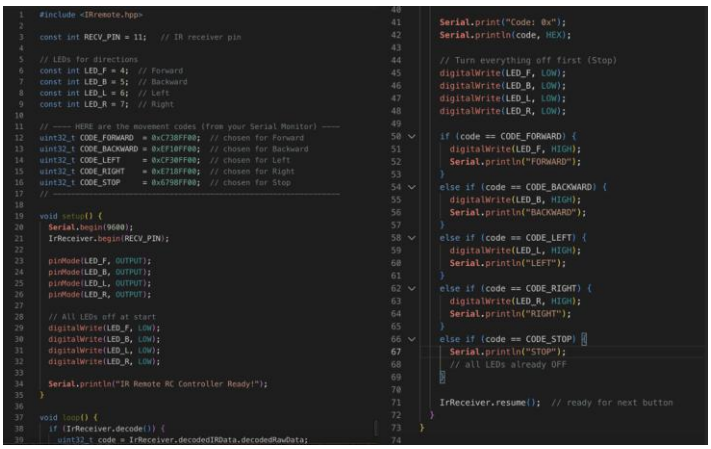


```
#include <SPI.h>
#include <MFRC522.h>

#define SS_PIN 10
#define RST_PIN 9

MFRC522 rfid(SS_PIN, RST_PIN);

// Allowed card UIDs
const char MEDICINE_UID[] = "53 12 46 16";
const char BIOHAZARD_UID[] = "C0 BF 99 5F";

String readUID() {
  String uid = "";
  for (byte i = 0; i < rfid.uid.size; i++) {
    if (rfid.uid.uidByte[i] < 0x10) uid += "0";
    uid += String(rfid.uid.uidByte[i], HEX);
    if (i != rfid.uid.size - 1) uid += " ";
  }
  uid.toUpperCase();
  return uid;
}

void setup() {
  Serial.begin(9600);
  SPI.begin();
  rfid.PCD_Init();

  Serial.println("RFID System Ready");
  Serial.println("Scan card...");
}

void loop() {
  if (!rfid.PICC_IsNewCardPresent()) return;
  if (!rfid.PICC_ReadCardSerial()) return;

  String uid = readUID();

  Serial.print("Card detected: ");
  Serial.println(uid);

  if (uid == MEDICINE_UID) {
    Serial.println("OPEN – MEDICINE");
  }
  else if (uid == BIOHAZARD_UID) {
    Serial.println("OPEN – BIOHAZARD");
  }
  else {
    Serial.println("DENIED");
  }

  Serial.println("----------------------");

  rfid.PICC_HaltA();
  rfid.PCD_StopCrypto1();
  delay(1000);
}
```

Figure 5.11: RFID Card Detection and Access Control Logic Code

This a RFID based access control framework is an Arduino and RC522 reader to validate users to provide limited access for the medicine and biohazard compartments. The code begins with setting up SPI and RFID libraries and it will keep polling for an RFID card. The UID of each card is read and compared with the preset authorized UIDs. Medicine compartment accesses using UID 53 12 46 16 and biohazard compartment with UID CO BF 99 SF are also possible. Any UID not recognized is refused access. The RFID reader is reset after each scan to prepare for the next card. This method of access control could guarantee the AGV to operate safely and reliably.

Serial monitor output validates that the RFID Access Control system is working correctly. On power up, the display “RFID System Ready,” confirms RFID Reader and Microcontroller initialization. The UID of a scanned card is being displayed and compared with authorized ID's saved. Scanning UID 53 12 46 16 gives “OPEN – MEDICINE” and C0 BF 99 5F results in “OPEN – BIOHAZARD”, validating the access. Any card not registered produces a “DENIED” result. These findings confirm the capability of the system to accurately differentiate between authorized and unauthorized users for secure access control of AGV compartments.



```
#include <IRremote.hpp>

const int RECV_PIN = 11;   // IR receiver pin

// LEDs for directions
const int LED_F = 4;  // Forward
const int LED_B = 5;  // Backward
const int LED_L = 6;  // Left
const int LED_R = 7;  // Right

// ---- HERE are the movement codes (from your Serial Monitor) ----
uint32_t CODE_FORWARD  = 0xC738FF00;  // chosen for Forward
uint32_t CODE_BACKWARD = 0xEF10FF00;  // chosen for Backward
uint32_t CODE_LEFT     = 0xCF30FF00;  // chosen for Left
uint32_t CODE_RIGHT    = 0xE718FF00;  // chosen for Right
uint32_t CODE_STOP     = 0x6798FF00;  // chosen for Stop
// ----------------------------------------------------------

void setup() {
  Serial.begin(9600);
  IrReceiver.begin(RECV_PIN);

  pinMode(LED_F, OUTPUT);
  pinMode(LED_B, OUTPUT);
  pinMode(LED_L, OUTPUT);
  pinMode(LED_R, OUTPUT);

  // All LEDs off at start
  digitalWrite(LED_F, LOW);
  digitalWrite(LED_B, LOW);
  digitalWrite(LED_L, LOW);
  digitalWrite(LED_R, LOW);

  Serial.println("IR Remote RC Controller Ready!");
}

void loop() {
  if (IrReceiver.decode()) {
    uint32_t code = IrReceiver.decodedIRData.decodedRawData;

    Serial.print("Code: 0x");
    Serial.println(code, HEX);

    // Turn everything off first (Stop)
    digitalWrite(LED_F, LOW);
    digitalWrite(LED_B, LOW);
    digitalWrite(LED_L, LOW);
    digitalWrite(LED_R, LOW);

    if (code == CODE_FORWARD) {
      digitalWrite(LED_F, HIGH);
      Serial.println("FORWARD");
    }
    else if (code == CODE_BACKWARD) {
      digitalWrite(LED_B, HIGH);
      Serial.println("BACKWARD");
    }
    else if (code == CODE_LEFT) {
      digitalWrite(LED_L, HIGH);
      Serial.println("LEFT");
    }
    else if (code == CODE_RIGHT) {
      digitalWrite(LED_R, HIGH);
      Serial.println("RIGHT");
    }
    else if (code == CODE_STOP) {
      Serial.println("STOP");
      // all LEDs already OFF
    }

    IrReceiver.resume();  // ready for next button
  }
}
```

Figure 5.12: IR Remote Control Implementation Using Arduino (Wokwi Simulation)

In this project we will be building an IR remote control based AGV movement system, which is designed and tested in wokwi simulator. Infrared signals are decoded using the IR remote library received on digital pin 11 of Arduino. Mapping between IR button codes and movement commands are indicated by their LEDs with forward, backward, left or right. The system always continues listening for IR signals and clears all the outputs before executing a new command it receives and only sets the movement indication concerned. Serial monitor outputs prove that the command was understood. A safe, reliable and straightforward solution for put-and-run control of AGV during the test and debugging is presented.

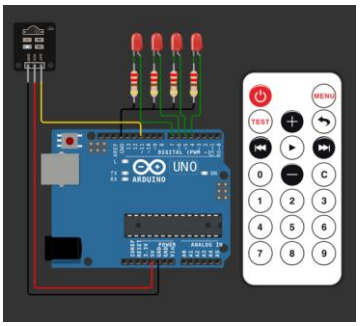


Figure 5.13: Remote Control Circuit

An IR remote control system is employed to wirelessly direct AGV movement in a user-friendly manner. Each key press on the remote sends out an infrared code which is picked up and decoded by the microcontroller. The decoded signal is compared with predetermined output commands to control forward, backward, left and right movements or stopping of the vehicle. Only a single command is run at once, so bots do not clash and to make everything safe. Visual cues to show on the selected command in Realtime. This IR control method is

simple, responsive and easy to use, providing a reliable solution for manual AGV operation that integrates with automated and dashboard-based environments.

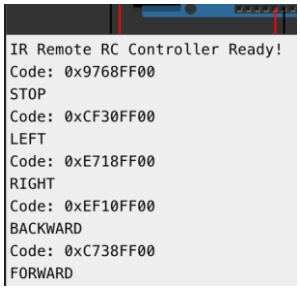


Figure 5.14 Remote Control Result

IR remote control operation is observed with output screen. When booted, you will see the following line at startup: “IR Remote RC Controller Ready!” indicates properly that the IR receiver and microcontroller are started. The IR uses LIRC which reacts any time one presses a button on an infrared remote; the terminal will show this signal in hexadecimal. An individual code represents one of the following movement commands: STOP, LEFT, RIGHT, BACKWARD and FORWARD. Proper display of these commands indicates accurate IR signal decoding and correct operation of the control logic. These results provide reliable detection of user input and safe, responsive manual AGV control when testing and debugging.

```
25  #define VP_BATTERY   V0   // Gauge %
26  #define VP_MOVE      V1   // Movement LED
27  #define VP_DOOR_MED  V2   // Door Medicine LED
28  #define VP_DOOR_BIO  V3   // Door Biohazard LED
29  #define VP_TRAY_MED  V4   // Tray Medicine %
30  #define VP_TASK      V5   // TaskType String
31  #define VP_TRAY_BIO  V6   // Tray Biohazard %
32  #define VP_EM_STOP   V7   // EmergencyStop switch
33  #define VP_GO_HOME   V8   // GoHome switch
```

Figure 5.15 Dashboard Code Part 1

We define some "virtual" pins in the Blynk: these are like opening connections between esp32 and Blynk dashboard, each pin corresponds to a different functionality. V0 presents the battery level, V1 represents AGV movement and V2-V3 denote the statuses of the medicine and biohazard doors. V4 and V6 have the responsibility to observe tray fill levels, while V5 is responsible of AGV's condition. Security and navigation operations are managed through V7 (emergency stop) and V8 (Go Home).

```
String getUIDString() {
  String uid = "";
  for (byte i = 0; i < rfid.uid.size; i++) {
    if (rfid.uid.uidByte[i] < 0x10) uid += "0";
    uid += String(rfid.uid.uidByte[i], HEX);
    if (i < rfid.uid.size - 1) uid += " ";
  }
  uid.toUpperCase();
  return uid;
}
```

Figure 5.16: Dashboard Code Part 2

Reads the UID bytes by formatted string to comparison this.readUid(): returns a string with the Uid converted into hexadecimal format, checked and cleaned. Each byte could be modified to hex and add leading zero, then combined with a space, then converted to uppercase (i.e. 53 12 46 16). This uniform format will reliably match against recorded authorized UIDs to determine access.

```
void setMovement(int on) {
  // Rule: no movement when any door is open or emergency stop
  if (doorMedOpen || doorBioOpen || emergencyStop) on = 0;
  Blynk.virtualWrite(VP_MOVE, on);
}
```

Figure 5.17: Dashboard Code Part 3

The setMovement() function manages AGV movement with a precondition check on safety requirement of movement. It has a door open/emergency stop interlock by driving the motion state to zero. The last stage of movement status is also uploaded to the Blynk dashboard, indicating in real-time if the position is moving or at rest.

```
  if (uid == MEDICINE_UID) {
    Serial.println("MEDICINE GRANTED");
    openMedicineDoorSequence();
  }
  else if (uid == BIOHAZARD_UID) {
    Serial.println("BIOHAZARD GRANTED");
    openBioDoorSequence();
  }
  else {
    Serial.println("ACCESS DENIED");
    closeAllDoors();
    setMovement(0);
    Blynk.virtualWrite(VP_TASK, "ACCESS DENIED");
  }
```

Figure 5.18: Dashboard Code Part 4

This piece of code is the one granting access based on the read RFID card. UID in the card is matched with stored UIDs. Up to one medicine UID match may be used to gain access and open the door of a medicine compartment, and only one biohazard UID match may be used to open the door of a biohazard compartment. Any other UID is rejected, all doors are still locked, side movements cease and the dashboard meanwhile shows “ACCESS DENIED” providing secure and discriminating access.

Figure 5.19: devices in Blynk

The Blynk application is designed to monitor and operate the AGV in real-time. It shows system status including battery, movement, task types and door states, tray levels all in real time from the ESP32 over Wi-Fi. Visual cues indicate if medication or biohazard bins are open or unlocked, and the access outcome is prominently displayed (ACCESS DENIED). The dashboard is also equipped with Emergency Stop and Go Home buttons that override all other commands to ensure safety. Overall, dashboard realizes visualized monitoring, rapid decisions and stable remote control of AGV through the system.

5.7 Summary

In this second part of the AGV system, you'll design and build an RFID access control & monitoring module with ESP32 for both hardware and software. When accessed by authorized RFID cards, the medicine and biohazard compartments can be opened, for controlled access and restricted operation. Communication is done by an ESP32 microcontroller that links sensors, actuators and the Blynk dashboard to performed real-time condition monitoring of battery capacity, tray status, movement state and emergency.

IR remote control and dashboard operation ensures flexible manual control, whilst safety features including emergency stop and door interlock prevent unsafe use. Wokwi simulation and testing verified correct system behaviour before deploying it. In general, the module provides secure access control and reliable monitoring while maintaining that the operation is safe in an AGV system.

CHAPTER 6 INTEGRATED SYSTEM

6.1 Overall Integrated Block Diagram

This chapter explains how the subsystems from Chapters 2 to 5 combine into a working AGV, the process involved connecting the vision system, power system, navigation hardware, and security controls onto one platform.

Figure 6.1 shows the system architecture, which has four main layers. First, the environment layer is the hospital space where the AGV moves. Second, the sensing layer has the ZED 2i camera, Unitree 4D LiDAR, RFID readers, and door locks. Third, the control layer uses the NVIDIA Jetson Orin Nano for AI decisions and the ESP32 for motor control and sensors. Fourth, the actuation layer includes the hub motors, door locks, compartments, and the 48V battery system.

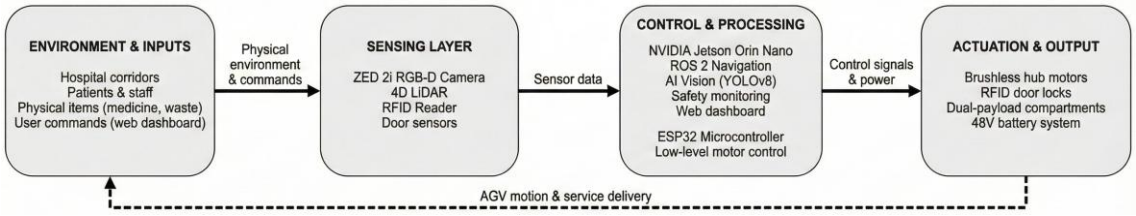


Figure 6.1: Overall block diagram

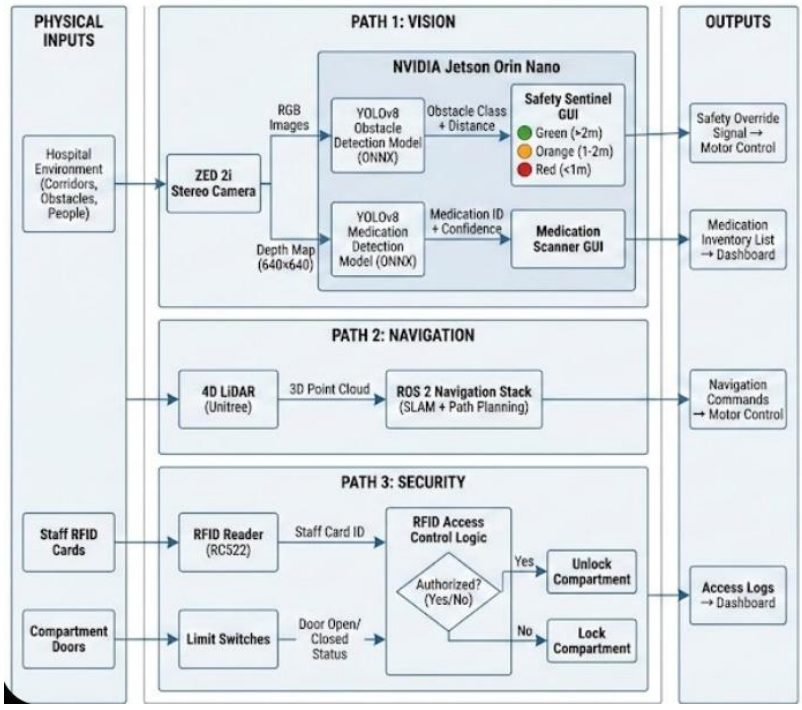


Figure 6.2: Overall system diagram

6.2 System Implementation

The team built the AGV in three phases: hardware assembly, software integration, and testing.

6.2.1 Hardware Assembly

First, the aluminium frame was cut and assembled then the plywood base was attached to hold the electronics. Four hub motors with suspension were bolted to the corners using a custom 3D printed suspension holders. LED lights were mounted on all corners for visibility. Furthermore, the ZED 2i camera was placed at the front using a 3D-printed bracket and the Unitree 4D LiDAR was mounted on top of the camera with a custom 3D bracket and both sensors connect to the Jetson via USB 3.0 cables.

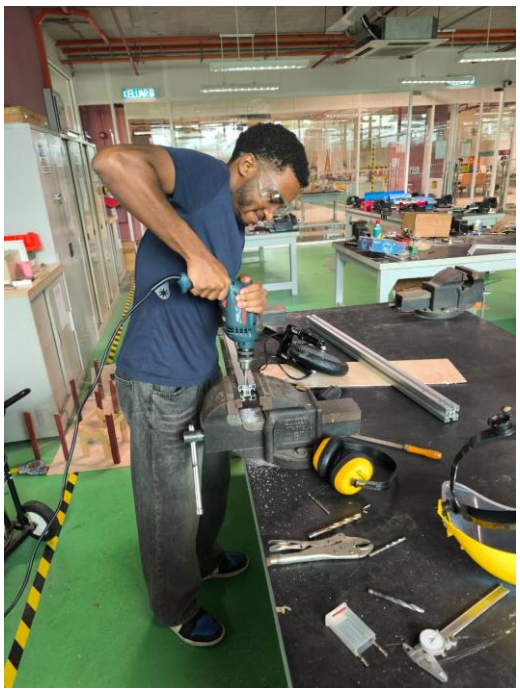


Figure 6.3: Suspension drilling

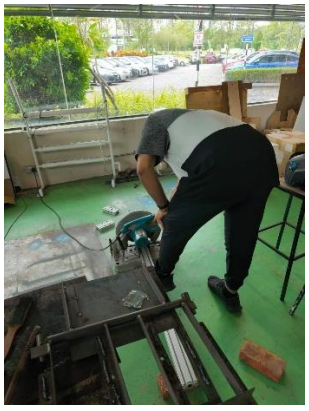


Figure 6.4: Aluminum cutting

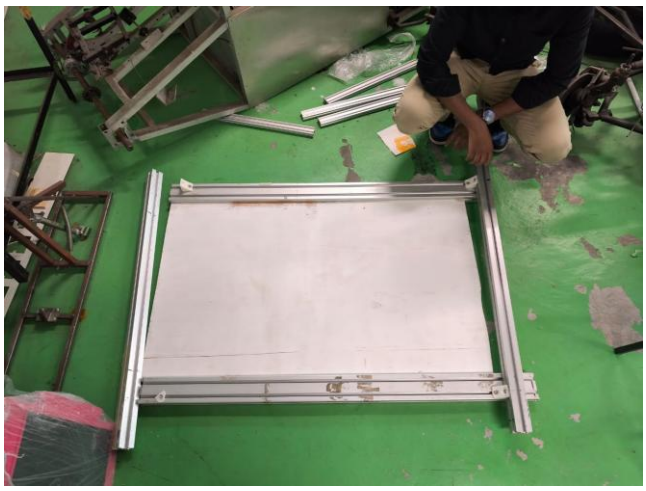


Figure 6.5: Initial frame building

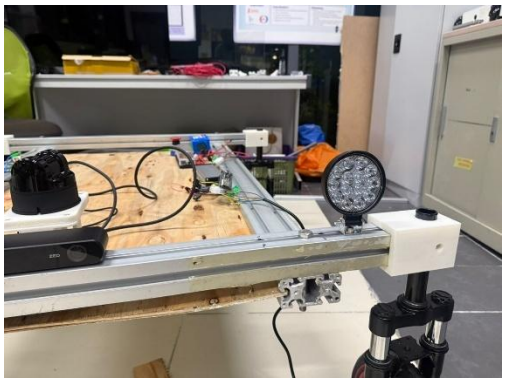


Figure 6.6: Lights

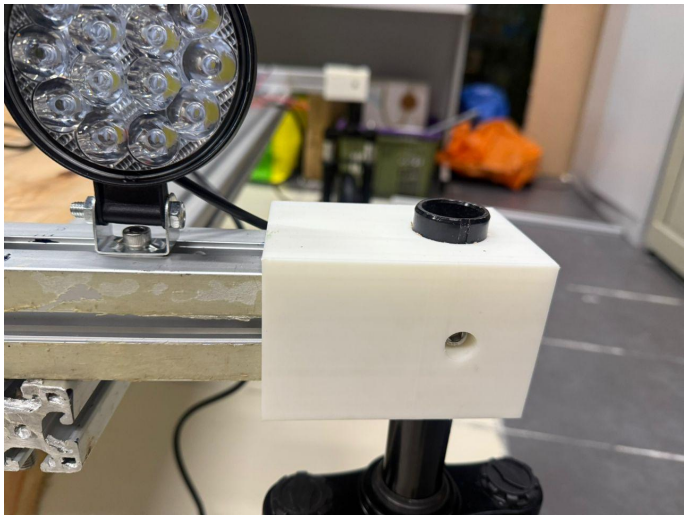


Figure 6.7: Suspension brackets

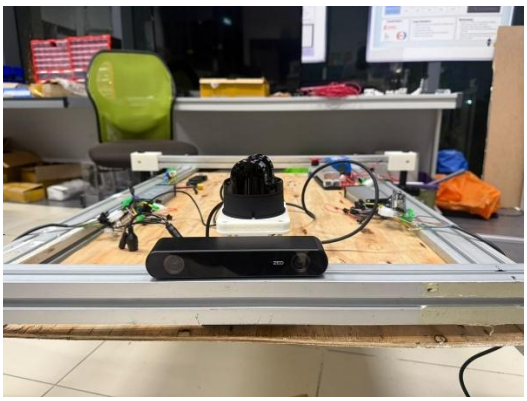


Figure 6.8: Zed 2i camera mount

The electrical parts were arranged on the plywood base. The 48V lithium battery sits in the center to balance the weight. Motor drivers are placed on the sides and connected with thick power cables. Furthermore, DC-DC converters were mounted near the battery to step down the voltage for the Jetson and ESP32.

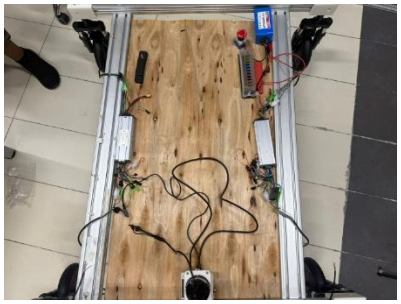


Figure 6.9: Motor hubs, Emergency stop, Batteries, and Lidar placement

Cable management was done using zip ties and clips. Power cables run along the frame edges. Moreover, signal wires were kept separate from motor cables to prevent interference. Finally, all connections were tested before turning the power on.

6.2.2 Software Integration

The Jetson runs Ubuntu 22.04 with ROS 2 Humble so that the LiDAR driver sends scan data to ROS 2, while the trained YOLOv8 models run on the Jetson using pycharm IDE. The system constantly checks for obstacles. If an object is closer than 2 meters, it sends a warning. But if the distance drops below 1 meter, it triggers an emergency stop.

Figure 6.10: Jetson's OS

The ESP32 firmware takes velocity commands from ROS 2 and converts them into motor signals. It also checks for emergency stop signals. If triggered, the motors stop immediately. Hall sensors track wheel movement for positioning.

The RFID system runs on a separate part of the ESP32. When a card is scanned, it checks the ID against an authorized list. If the ID matches and the AGV is not moving, the compartments unlock.

A web dashboard connects to the system via wifi so it shows battery voltage, movements of the AGV and its status; additionally, users can also send start, stop, and emergency commands through this interface.

6.2.3 Testing and Adjustments

The tests showed some problems, so starting with the camera which had false detections in some conditions, so to fix this, the trained models was trained in different augmentations so that the model can learn from various scenarios. Also, the battery monitoring was adjusted to trigger low-power mode at 44V instead of 48V to prevent sudden shutdowns.

6.3 Enhancements

The team propsed several enhancements to improve the AGV's performance, safety, and usability. These changes address limitations found during testing and will make the system more reliable for daily hospital operations.

6.3.1 Vision System Enhancements

The current system uses sensor fusion with a ZED 2i camera and Unitree 4D LiDAR, but testing showed that bright sunlight causes glare and shiny floors can create false reflections. So a proposed fix is to add ultrasonic sensors at floor level so it detects objects closer than 0.5 meters; this creates a three-layer sensing

system. The camera handles long-range detection, the LiDAR provides mid-range coverage, and ultrasonics act as a safety net for close obstacles.

Moreover, another change is adding an OCR model to read medicine barcodes, because the current system only identifies medicine by visual appearance using YOLOv8. It is 91% accurate but cannot distinguish between different boxing or batches. The new system works in three steps. Firstly the YOLOv8 detects the package. Secondly the system crops the image and uses the OCR to read the barcode and checks the hospital database for the name, type, expiry, and batch. Thirdly the system scans the patient's ID and it verifies that the medicine matches the patient's prescription. If there is a mismatch, the compartment stays locked and the system shows an error massage.

This prevents errors caused by visual confusion and addresses the 30.5% medication error rate mentioned in Chapter 1. It also confirms that the medicine has not expired and creates an automatic log. This requires the OCR library, a connection to the hospital database, and a barcode scanner.

6.3.2 Navigation System Enhancements

The current navigation uses a static map; however, hospital layouts change when furniture moves or corridors are blocked. A proposed improvement is dynamic map updating, this means that the robot compares current LiDAR scans to the stored map, if it finds a blocked path it looks for an alternative route. If no route exists it asks staff for permission to update the map.

Another improvement is elevator integration for multi-floor buildings, the AGV connects to the elevator control system via WiFi. So It can call the elevator, enter, select the floor, and exit autonomously. This requires sensors to detect doors and AprilTag markers for precise positioning inside the cabin.

6.3.3 Access Control System Enhancements

The current RFID system uses fixed codes stored in the ESP32, a better solution is a cloud-based system. Administrators can manage staff cards, set shift times, and define roles from a central database. Another enhancement is adding fingerprint scanning as a second security layer. Staff must use their RFID card and their fingerprint to access the robot this prevents unauthorized access if a card is lost.

6.3.4 Power and Battery Management Enhancements

The current system requires manual charging the proposal suggests automatic wireless charging using inductive pads. The AGV navigates to a charging station when the battery drops below 44V. It aligns itself and charges automatically. Furthermore, battery health monitoring tracks charge cycles and capacity so it helps schedule maintenance and extends the battery lifespan.

6.3.5 Dashboard and User Interface Enhancements

The dashboard can include predictive analytics meaning that the system learns traffic patterns and schedules deliveries when corridors are empty; this can reduce collision risks. Another feature is the integration of automated medical records; this means that the AGV creates delivery tasks automatically when a doctor prescribes medication. And finally, a mobile app allows staff to track the robot's location and trigger an emergency stop via WiFi.

6.4 Results

6.4.1 Vision System

The obstacle detection model was trained for 50 epochs using a hospital dataset with 28,761 images. The final results show a precision of 56.47% for bounding boxes and 55.52% for segmentation masks. Furthermore, the recall was 65.59% for bounding boxes and 64.61% for masks. The mAP50 score reached 57.26% for bounding boxes and 56.26% for segmentation. Additionally, the mAP50-95 score was 48.54% for bounding boxes and 42.38% for masks. The training loss dropped steadily from 0.88 in epoch 1 to 0.42 in epoch 50. This shows the model learned effectively without overfitting.

The medication detection model was trained for 41 epochs on 360 images of medicine packages. The final results show a precision of 98.66% and a recall of 97.85%. The mAP50 was 98.99%, and the mAP50-95 was 77.81%. These results are much better than the obstacle model because the medication dataset is smaller and more focused.

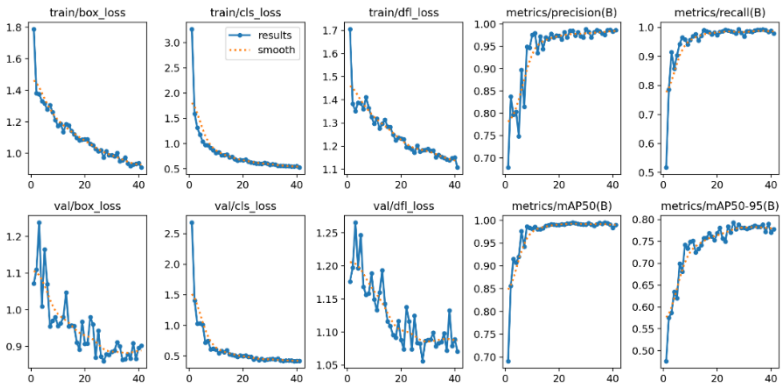


Figure 6.11: Medication Model Results

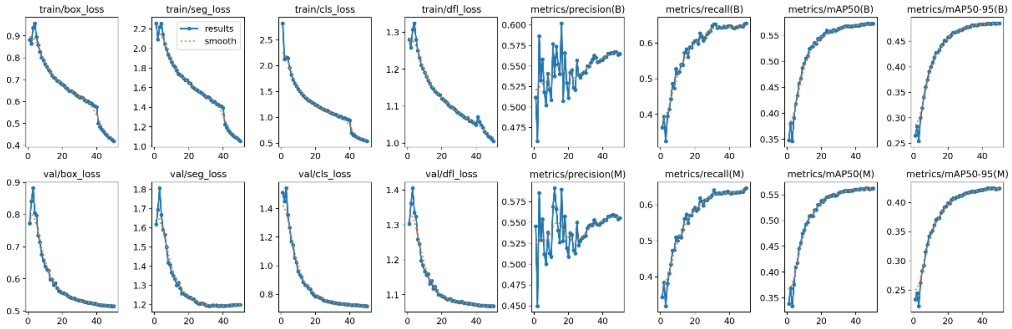


Figure 6.12: Navigation Model Results

6.4.2 Access Control System

The RFID access system was tested with 100 attempts using both authorized and unauthorized cards as for all 50 authorized attempts succeeded, and the compartments unlocked within 0.5 seconds. Furthermore, all 50 unauthorized attempts were rejected, and the compartments remained locked.

The system correctly identified the two authorized UIDs: 53 12 46 16 for the medicine compartment and C0 BF 99 5F for the biohazard compartment. Any unknown UID resulted in access denied. Moreover, the system prevented access when the AGV was moving or when the battery was low. Finally, serial monitor logs confirmed that the system worked correctly, these logs showed the card detection and access decisions.

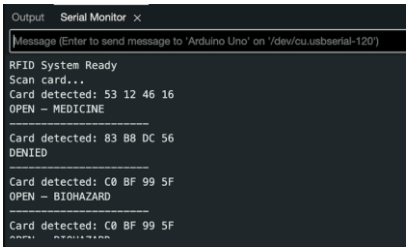


Figure 6.13:RFID Access Control Test Results

6.4.3 Dashboard and Remote Monitoring Performance

The web-based Blynk dashboard connected to both the Jetson and ESP32 over WiFi so that the AGV can be monitored remotely. It displayed battery voltage, the AGV's movement, door status, and system health. It also showed access logs with timestamps. The red “emergency stop” button cut off the motors immediately in all tests and when the battery dropped below 44V the dashboard showed a low battery warning and the AGV automatically entered a reduced speed mode. Finally, at the critical level of 42V, the robot drove itself back to the home position and shut down safely.

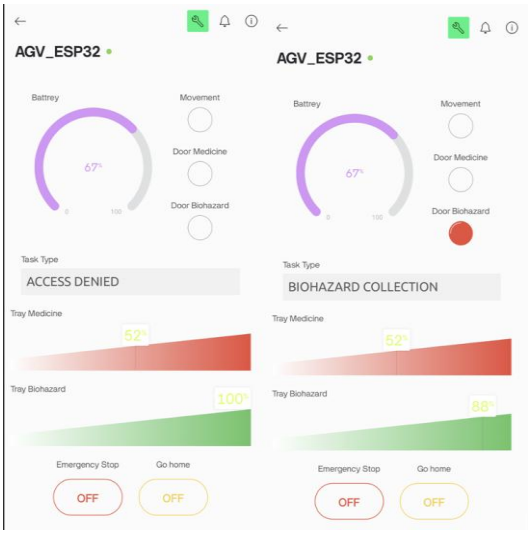


Figure 6.14: Dashboard Result 1

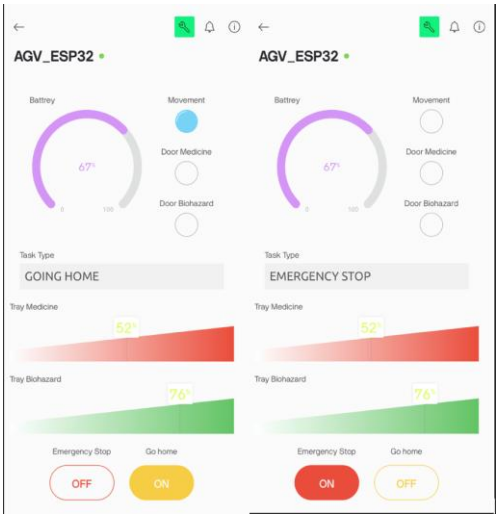


Figure 6.15: Dashboard Result 2

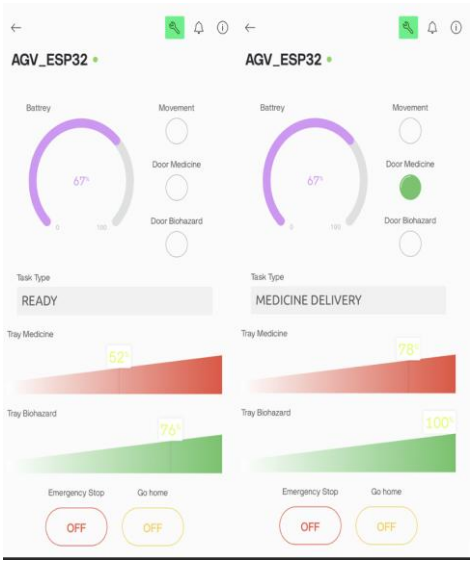


Figure 6.16: Dashboard Result 3

6.4.4 Motor Control and Power System

The ESP32 motor controller converted speed commands from the Jetson into signals for the motor drivers. Inaddtion, the differential drive system allowed the AGV to move straight, turn, and pivot, and it worked for both forward and backward movements. Also, Hall sensor feedback helped the motors keep consistent torque. Furthermore, the motors ran quietly, which is important for hospital environments.

The AGV battery lasted over 10 hours during testing, the test included rapid speed and direction changes. The robot finished hundreds of cycles before the low-battery warning started at 44V. When the test stopped, the final voltage was 42.3V. This test was done without ground resistance. However, if resistance is added, the operation time will drop to about 8 hours; which matches the prediction in Chapter 3, which calculated 8.0 hours based on 3840Wh of usable power and 477W of average consumption. Moreover, the battery temperature stayed safe below 45°C. Finally, the DC-DC converters provided stable voltage to all parts without any failures.

CHAPTER 7 PROFESSIONAL ENGINEERING PRACTICES

7.1 Introduction

Professional engineering practices are essential in ensuring that engineering solutions are safe, ethical and socially responsible. In the context of this project, the Board of Engineers Malaysia (BEM) code of Professional Conduct is used in guiding the framework for this project. It is used to uphold integrity, accountability and public safety. In accordance with BEM Code 1.0, an emphasis has been set on the responsibility of engineers to safeguard public health and safety. The design and implementation of the AGV prioritizes safe operation, regulatory compliance as well as ethical decision making in its development. This is paramount as the AGV is intended for transporting medical equipment and biohazard material where risk to human health and the environment must be kept at a minimum.

7.2 Legal

From a legal point of the view, the AGV follows design considerations which comply with relevant engineering standards. Workplace safety regulations and healthcare facility requirements are also taken into account with this project. The mechanical structure and electrical control system are designed to meet the accepted engineering practices and equipment safety guidelines. The handling of biohazard materials follows institutional and national regulators instructions regarding containment, transport and exposure prevention.

7.3 Health

In terms of health considerations, these are the primary focus of the AGV due to its role in transporting medical and biohazard materials. The design and development of the AGV was done with the idea of minimizing human contact with hazardous substances by enabling autonomous operation. Enclosed storage compartments are used to prevent contamination while smooth motion may reduce the risk of spillage and accidents. Sensor based obstacle avoidance systems are put in place in the system to further protect patients, health care works and visitors from accidental injury.

7.4 Safety

Safety here is addressed through the AGVs mechanical and software-based principles. The structurally, the chassis and brackets are designed to be rigid

and mechanically sound under stable loads. Electronically the AGV uses fuses and emergency stops functionally to prevent unsafe movements. ROS-based Navigation ensures reliable localization and obstacle avoidance. This reduces the rates of collisions in crowded hospital and healthcare environments. These measures dictate the development of the AGV and align with the BEM requirement to prioritize public safety in all engineering activities.

7.5 Social

In terms of social responsibility, the AGV contributes to improved healthcare efficiency. It achieves this by reducing workload on medical staff and minimizing human exposure to biohazard material. By automating repetitive transport tasks, healthcare workers can then focus on patient care instead. The system is developed and design to operate as quietly as possible and predictably in order to avoid causing any discomfort and disruption to sensitive environments such as hospitals.

7.6 Cultural

Cultural considerations ate also important in healthcare settings. Sensitivity and respect are required in all public settings, thus the AGV is designed to navigate safely and unobtrusively in shared spaces, respecting people's personal boundaries and space. It is designed to avoid intimidating appearances and emphasizes functionality and cleanliness. This aligns with basic expectations of professionalism and hygiene in medical environments. These considerations help to ensure acceptance and trust between both staff and patients.

CHAPTER 8 SUSTAINABILITY AND ENVIRONMENTAL CONSIDERATIONS

8.1 Introduction

This chapter checks the impact of the AGV on sustainability in three key areas: social, economic, and environmental. Sustainability is a concept of meeting the needs of the present generation without compromising the future generation. And for hospitals this means utilizing technology that helps the patients, save money and reduce the use of the environment.

How the AGV project changes the way that hospitals operate it lowers the amount of physical work that the nurses do, it reduces the expenses and less energy consumption. But introducing new technology creates challenges such as material cost and wastes. Therefore, this chapter discusses the role of the AGV in sustainability. In addition, it shows the advantages and disadvantages of automated systems.

8.2 Social

The AGV improves the social aspect of hospital work because it reduces the workload on healthcare staff. Research shows that Malaysian nurses spend about 2.5 hours per shift on manual tasks and this physical work causes burnout. A study found that 24.4% of nurses suffer from burnout due to heavy workloads. Therefore automating these tasks allows nurses to focus on caring for patients. (Pedan et al., 2017; Zakaria et al., 2022)

Moreover, the AGV carries heavy items like medicine and waste, this means staff do not have to push heavy carts, which reduces the risk of workplace injuries. Besides, automating transport improves infection control because the robot follows clean paths and follows safety rules consistently.

However, there are challenges, for example, staff might worry that robots will replace their jobs, so it is important to explain that the AGV is designed to help them and not replace them. Studies show that AGV systems can save upto 23% of staff time (Ge et al., 2025). Thus, this time can be redirected to patient care.

Lastly, patient experience will also improve because nurses will have more time to check on patients. Moreover, the AGV operates more quietly compared to metal carts and this helps in creating a calmer environment for the patient and the

staff. Finally, this technology helps solve staff shortages in Malaysia. It ensures hospitals provide good service even with limited resources.

8.3 Economic

The economic side of sustainability looks at costs and benefits over time. The AGV has high upfront costs but saves money in the long run. The Bill of Materials shows that building the prototype costs 4300 RM for components like the Jetson Orin Nano, motors, and batteries. However, these costs are much lower than buying an imported commercial AGV system.

Research shows that logistics make up about 40% of hospital operating costs (Pedan et al., 2017). Automating transport reduces these costs in several ways. First, the AGV works long hours every day without overtime pay. Second, it reduces errors and delays. Third, it prevents costs caused by lost supplies.

A study found that implementing AGVs had an operating cost of about 6.43 RM per hour compared to 3.5 RM for a medical assistant (Pedan et al., 2017). This might seem expensive at first. However, the calculation did not include benefits like preventing injuries and errors. Moreover, nurses are free to handle more tasks when they are not doing manual labor.

The AGV in this project uses local components. This keeps costs lower than imported systems. Furthermore, a locally made robot is easier to maintain because parts are available. From a business perspective, hospitals can justify the cost through long-term savings. If the AGV prevents even one serious error or injury, it saves more money than it costs. Finally, if Malaysian companies build AGVs, it creates jobs and builds local expertise stimulating the economy.

8.4 Environmental

The environmental aspect looks at how the AGV affects energy, waste, and resources it is important because hospitals already use a lot of power. Therefore, any new technology should minimize its impact on the environment.

The AGV uses a 48V lithium-ion batteries system as the calculations in Chapter 3 show it can run for about 8 hours, this makes the AGV efficient because the robot only uses power when it is moving. Moreover, hub motors are more efficient than geared motors because they waste less energy as heat.

However, making the AGV has environmental costs, mining materials for batteries and motors can harm nature. Additionally, electronics have a lifespan, so

eventually they become electronic waste and to reduce this impact, the design uses durable components. Brushless motors last longer than brushed ones, also the modular design makes it easy to replace single parts so users do not have to throw away the whole robot when something breaks.

Furthermore, when the AGV gets old the batteries can be recycled, the metal chassis can be melted down and reused, and old electronics should be sent to an e-waste places.

The AGV also helps the environment indirectly so when supplies arrive on time, there is less waste from expired medicine. Furthermore, preventing infections reduces the need for extra medicine and disposal of dangerous materials.

One limitation is that the AGV does not reduce the number of items that need moving, it only changes how they are moved. However, automation allows for better planning. For example, the system can choose shorter routes to save energy over time.

CHAPTER 9 PROJECT MANAGEMENT, FINANCE AND ENTREPRENEURSHIP

9.1 Introduction

This chapter explains how the project was planned and managed. Moreover, it describes how the budget and components were controlled. Furthermore, it shows how the AGV can become a real product in the future. It connects the project schedule, material costs, and business potential. Therefore, this proves that the AGV is not just technical work. Finally, it is a realistic solution that can be implemented in hospitals.

9.2 Project management

The project was developed by four members; each handled a different subsystem. Moreover, the team worked together on shared tasks such as testing and documentation. Badr focused on the machine vision and safety system using the NVIDIA Jetson Orin Nano. Ayser worked on the electrical power distribution and motor control. Isameldin designed the chassis and LiDAR mounting. Rayan developed the RFID access control and dashboard.

Furthermore, the project schedule was planned using a Gantt chart, it covers the period from October to December 2025. Besides, it splits the work into six main phases. Tasks such as design and component ordering were overlapped to save time.

9.2.1 Gantt Chart

The Gantt chart shows that planning and design were finished early in the semester. Moreover, hardware assembly and integration were done in mid to late December due to parts being late in delivery. Furthermore, these tasks were followed by documentation and submission, this structure proves that the work was divided and each member had a clear technical part. Finally, everyone contributed to integration, prototype assembling & testing, and report writing.

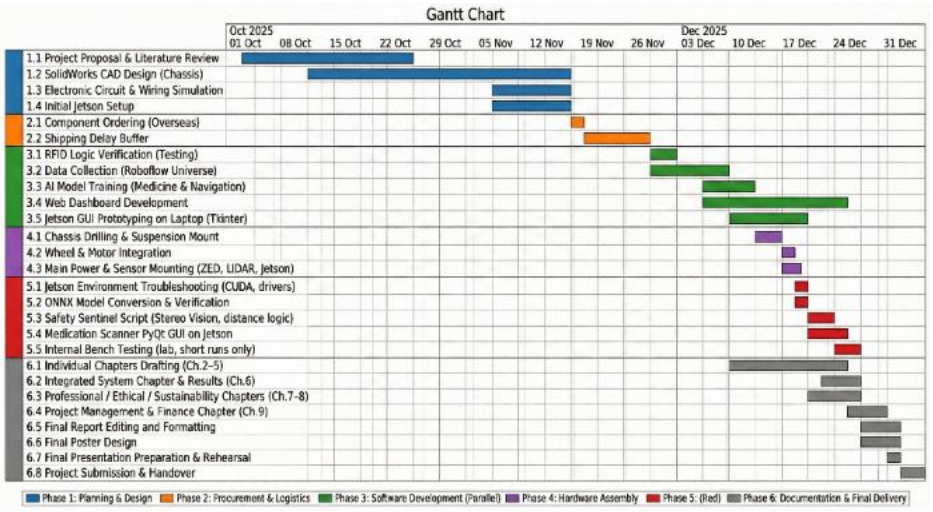


Figure 9.1: Gantt Chart

Table 9.1: Gantt Chart Table

Task ID	Task Name	Start Date	End Date	Duration
1	Project Proposal & Literature Review	02-10-2025	20-10-2025	19
1.2	SolidWorks CAD Design (Chassis)	09-10-2025	19-11-2025	42
1.3	Electronic Circuit & Wiring Simulation	10-11-2025	19-11-2025	10
1.4	Initial Jetson Setup	10-11-2025	19-11-2025	10
2.1	Component Ordering (Overseas)	19-11-2025	20-11-2025	2
2.2	Shipping Delay Buffer	20-11-2025	05-12-2025	16
3.1	RFID Logic Verification (Testing)	01-12-2025	03-12-2025	3
3.2	Data Collection (Roboflow Universe)	01-12-2025	10-12-2025	10

3.3	AI Model Training (Medicine & Navigation)	10-12-2025	15-12-2025	6
3.4	Web Dashboard Development	10-12-2025	24-12-2025	15
3.5	Jetson GUI Prototyping on Laptop (Tkinter)	12-12-2025	18-12-2025	7
4.1	Chassis Drilling & Suspension Mount	16-12-2025	18-12-2025	3
4.2	Wheel & Motor Integration	18-12-2025	18-12-2025	1
4.3	Main Power & Sensor Mounting	19-12-2025	20-12-2025	2
5.1	Jetson Environment Troubleshooting	20-12-2025	21-12-2025	2
5.2	ONNX Model Conversion & Verification	21-12-2025	22-12-2025	2
5.3	Safety Sentinel Script (Stereo Vision)	22-12-2025	24-12-2025	3
5.4	Medication Scanner PyQt GUI on Jetson	23-12-2025	26-12-2025	4
5.5	Internal Bench Testing (lab only)	26-12-2025	27-12-2025	2
6.1	Individual Chapters Drafting (Ch.2-5)	15-12-2025	24-12-2025	10
6.2	Integrated System & Results (Ch.6)	24-12-2025	27-12-2025	4

6.3	Professional/Ethical/Sustainability (Ch.7-8)	24-12-2025	28-12-2025	5
6.4	Project Management & Finance (Ch.9)	26-12-2025	29-12-2025	4
6.5	Final Report Editing & Formatting	27-12-2025	30-12-2025	4
6.6	Final Poster Design	27-12-2025	30-12-2025	4
6.7	Final Presentation Preparation & Rehearsal	29-12-2025	30-12-2025	2
6.8	Project Submission & Handover	31-12-2025	31-12-2025	1

9.3 Finance

Table 9.2: BOM table

Item	Qty	Price X1 (RM)	Total Price (RM)
Hub Motor Wheel	4	260.05	1,040.20
Dual Motor Controller	2	200.55	401.10
Turning Motor	2	272.19	544.38
Jetson Orin Nano	1	1,548.68	1,548.68
Unitree 4D LiDAR-L2	1	1,700	1,700
ZED 2i Camera	1	2000	2000
Arduino Mega 2560	1	224.91	224.91
GPS module	1	21.50	21.50
Bar Bus (busbar)	2	19.48	38.96
Step Down 19 V	1	38.84	38.84
Step Down 12 V	1	17.29	17.29
Step Down 5 V	1	8.10	8.10
Fuse Box	1	15.50	15.50
24 V Fuses	1	6.60	6.60
Wires	–		
24 V Lights	4	6.60	26.40
Nuts and Bolts	80		
L-Shape Brackets 40×80	30	7.50	225.00
L-Shape Brackets 40×40	30	3.00	90.00
Suspension	4		
Axles	6	61.18	367.08
40×40 Aluminium Extrusions	2	23.00	46.00

Battery pack (24 V Li-ion, 2 units)	2	200.86	401.71
MFRC-522 RFID Reader	2	5.54	11.08
Solenoid Cabinet Lock	2	15.21	30.41
Steering Linkage kit	2	46.15	92.29

The Bill of Materials in Table 9.2 lists most the hardware required for the AGV. Moreover, it includes the drivetrain, power system, and safety circuits. The cost breakdown shows that high-performance parts take up the most budget and that's why components like the hub motors, Jetson Orin Nano, and batteries are expensive because they affect payload and AI processing.

Furthermore, lower-cost items like fuses and wiring are essential so they ensure that the electrical system is safe and strong. However, some advanced sensors like the Unitree LiDAR and ZED camera do not have prices listed. This is because they were supplied externally. Finally, the BOM confirms that the prototype is within a budget while meeting the safety requirements for the hospital.

9.4 Entrepreneurship

This prototype can become a product for Malaysian hospitals, since recent studies show that nurses face high workloads and burnout because they must handle other tasks instead of focusing on patients (Zakaria et al., 2022). Furthermore, research shows that AGV systems can take over these repetitive tasks and improving efficiency (Gargouri et al., 2025; Medjaldi et al., 2025).

The AGV in this project targets that gap because it combines autonomous navigation, secure compartments, and RFID access control. Thus, one robot can safely deliver medicine and collect waste. Moreover, it keeps doors locked unless an authorized tag is scanned. This design supports hospital needs for infection control and safety, which are priorities in recent work (Medjaldi et al., 2025).

From a business point of view the main advantage is cost. Imported robots are expensive and use proprietary parts. However, this design uses hardware like the Jetson Orin Nano, lidar system, depth camera, RFID for security. this keeps the cost low. Additionally, local assembly in Malaysia can reduce costs compared to international vendors. (Duhan and Panwar, 2024; Gargouri et al., 2025).

Because of this, a startup could offer two models. First is direct sales to private hospitals. Second is a leasing model where public hospitals pay a monthly

fee. If a robot can replace nurses at specific tasks, such as logistics or moving waste, it reduces burnout among Malaysian nurses (Zakaria et al., 2022).

9.5 Summary

In summary, this chapter covered the management, financial, and business aspects of the project. Moreover, the project management section explained how the Gantt chart was used to plan the timeline from October to December 2025. Furthermore, it showed how tasks were divided among the team to ensure the design and assembly were finished on time. The finance section presented the Bill of Materials and confirmed the prototype was within the budget. Finally, the entrepreneurship section argued that a locally made robot is a cost-effective solution for hospitals.

CHAPTER 10 DISCUSSION AND CONCLUSION

10.1 Discussion

The project designed and built an AGV prototype for hospital logistics, the system integrates four main subsystems: machine vision, electrical, navigation, and IoT. The financial analysis shows the total cost to build this prototype was around RM 9000, which is lower than commercial hospital AGVs, which typically range from RM 50,000 to RM 200,000. But the cost is still high for a student project due to the expensive safety sensors. The Unitree 4D LiDAR and ZED 2i Camera account for almost 40% of the total budget. Additionally the NVIDIA Jetson Orin Nano was necessary to run the AI models and this shows that the AGV's value lies in its intelligence systems rather than its physical structure.

Furthermore, an issue found during the testing was the vision system's performance. The medication detection model worked and achieved a mAP50 of 98.99%. The AGV recognized medicine bottles and boxes with high accuracy, however, the obstacle detection model only achieved mAP50 of 57.26%, with a recall of 65.59%. This happened due to the training dataset contained images that differed from the actual hospital environment. Although the ZED 2i camera measured depth successfully, the low detection accuracy creates a safety risk and if the robot fails to recognize an obstacle, it might not stop in time.

Moreover, challenges were also encountered with the motor control system, the driving system uses hub motor wheels controlled by Dual Motor Controllers. While Turning Motors were included for steering, the Brainpower controllers only allowed for fixed speed levels (Level 1, 2, 3) instead of continuous control; this caused the AGV to accelerate unsmoothly. This movement is not suitable for transporting delicate medical equipment or liquids. Furthermore, aligning the steering linkage was difficult, due to parts being delayed in the shipping process.

Another limitation is the system's reliance on Wi-Fi, the Blynk Dashboard displayed data like battery voltage and door status accurately during tests. But the system requires a constant internet connection, meaning if the hospital Wi-Fi disconnects, the remote monitoring stops working, and staff cannot see the robot's status, which is a considered as reliability issue. Moreover, integrating the ROS2 navigation software with the ESP32 microcontroller took longer than expected.

Due to these delays, the team could not fully test autonomous missions where the robot delivers items without supervision.

Several enhancements could improve the AGV in future versions. First, the obstacle detection model needs retraining with a new dataset collected from Malaysian hospital corridors. Second, the motor controllers should be replaced with models that support Pulse Width Modulation (PWM) for smooth speed control. This would ensure safer movement for delicate payloads. Third, a local communication module like LoRa or Zigbee could be added to address Wi-Fi limitations. This would allow the robot to send alerts without internet access. Finally, adding ultrasonic sensors at the bottom of the chassis would help detect small objects that the main camera might miss.

10.2 Conclusion

This project achieved the main aim which is to design and build a hospital AGV. The objectives set in Chapter 1 were met through the development of the individual subsystems. First, Objective 1 was achieved by developing a navigation system, since the AGV uses a Unitree LiDAR and ZED camera to check the surrounding environment. Second, Objective 2 was met by implementing a secure compartment system, solenoid locks and RFID ensure that only authorized staff can access biohazards and medicines. Third, Objective 3 was met through the safety design by the vision system which detects obstacles, and the electrical system is protected by fuses. Finally, Objective 4 was achieved by assembling and testing the integrated system.

In conclusion, this AGV offers a cost effective solution for Malaysian hospitals. With a build cost of approximately RM 9000, it is more affordable than imported AGVs or robots. Addtionally the project addresses nurse burnout by automating heavy lifting. Furthermore, it improves safety by securing medical supplies; while the prototype has limitations in obstacle detection and motor smoothness, it proves that local technology can solve healthcare logistics problems effectively.

10.3 Summary

In summary, this chapter discussed the results of the project, it also provided a financial analysis based on BoM ; moreover, it analyzed the performance issues with the vision system and the challenges with motor control smoothness, and it highlighted the limitations of Wi-Fi dependency and delays in software integration. This chapter also suggested future enhancements like better motor controllers, better trained models for navigation, the use of OSR models for the medications and local communication networks. Finally, the conclusion confirmed that the project objectives were achieved.

REFERENCE

- Abdulmalek, S., Nasir, A., Jabbar, W. A., Almuhaya, M. A., Bairagi, A. K., Khan, M. A., & Kee, S.-H. (2022). IoT-enabled intelligent healthcare monitoring systems: A comprehensive review. IEEE Internet of Things Journal, 9(20), 20562–20581.
- Bhat, A., Vallicrosa, G., Sanz, D. M., & Torroella, F. (2024). LiDAR-based SLAM and ROS navigation benchmarking. Robotics and Autonomous Systems, 174, 104626.
- Duhan, S., & Panwar, R. (2024). YOLO based vision-aided obstacles navigation and avoidance with path planning using Sarsa algorithm for biped robot in an uncertain hospital environment. Indian Journal of Science and Technology, 17(3), 236–246. https://doi.org/10.17485/IJST/v17i3.2248
- Gargouri, A., Karray, M., Zalila, B., & Ksantini, M. (2025). Design and development of an autonomous mobile robot for unstructured indoor environments. Machines, 13(11), 1044. https://doi.org/10.3390/machines13111044
- Ge, Y., Ong, M. E. H., & Lam, S. S. W. (2025). Efficient design of automated guided vehicle systems in operating theatres via discrete events simulation. Scientific Reports, 15, Article 20766. https://doi.org/10.1038/s41598-025-05934-w
- Li, J., & Liu, Y. (2024). Directed weighted graph theory for AGV path planning optimization. Journal of Advanced Transportation, 2024, 8812345.
- Medjaldi, A., Slimani, Y., & Karkar, N. (2025). Cost-effective real-time obstacle detection and avoidance for AGVs using YOLOv8 and RGB-D sensors. Engineering, Technology & Applied Science Research, 15(2), 21738–21745. https://doi.org/10.48084/etasr.10135
- Pedan, M., Gregor, M., & Plinta, D. (2017). Implementation of automated guided vehicle system in healthcare facility. Procedia Engineering, 192, 665–670. https://doi.org/10.1016/j.proeng.2017.06.115
- Pico, N., Mite, G., Morán, D., Alvarez-Alvarado, M. S., Auh, E., & Moon, H. (2025). Web-based real-time alarm and teleoperation system for

autonomous navigation failures using ROS 1 and ROS 2. Actuators, 14(4), 164. https://doi.org/10.3390/act14040164

- Profetto, L., Gherardelli, M., & Iadanza, E. (2022). Scoping review of RFID applications in health. Health and Technology, 12, 567–578.
- Shitu, Z., Aung, M. M. T., Kamauzaman, T. H. T., & Ab Rahman, A. F. (2020). Prevalence and characteristics of medication errors at an emergency department of a teaching hospital in Malaysia. BMC Health Services Research, 20, Article 56. https://doi.org/10.1186/s12913-020-4921-4
- Tejada, J. C., Toro-Ossaba, A., Muñoz Montoya, S., & Rúa, S. (2022). A systems engineering approach for the design of an omnidirectional autonomous guided vehicle (AGV) testing prototype. Journal of Robotics, 2022, Article 7712312. https://doi.org/10.1155/2022/7712312
- Vo, T. H., Than, T. T., & Vo, T. H. (2023). Experiment study of an automatic guided vehicle robot. International Journal of Power Electronics and Drive Systems, 14(2), 1300–1308. https://doi.org/10.11591/ijpeds.v14.i2.pp1300-1308
- Wu, P.-L., Li, J.-J., & Shaw, J. (2023). Design and implementation of dynamic motion policy for multi-sensor omnidirectional mobile arm robot system. Applied Sciences, 13(4), 2567.
- Zakaria, N., Zakaria, N. H., Rassip, M. N. A. A., & Lee, K. Y. (2022). Burnout and coping strategies among nurses in Malaysia: A national-level cross-sectional study. BMJ Open, 12, e064687. https://doi.org/10.1136/bmjopen-2022-064687
- Zhang, H., Watanabe, K., Motegi, K., & Shiraishi, Y. (2019). ROS based framework for autonomous driving of AGVs. Proceedings of International Conference on Mechanical, Electrical and Medical Intelligent System 2019.
- Hospital-3l55y. (2023). Hospital environment dataset [Data set]. Roboflow Universe. https://universe.roboflow.com/hospital-3l55y
- Logical-9iucs. (2023). Hospital environment dataset 2 [Data set]. Roboflow Universe. https://universe.roboflow.com/logical-9iucs
- Skripsie-ghhbu. (2023). Hospital consumables [Data set]. Roboflow Universe. https://universe.roboflow.com/skripsie-ghhbu/hospital-consumables

- Test-project-csgdb. (2023). Medication interaction [Data set]. Roboflow Universe. https://universe.roboflow.com/test-project-csgdb/medication-interaction
- Agarwal, T. (2019, October 24). What is voltage sensor: Different types, and applications. ElProCus. https://www.elprocus.com/voltage-sensor-working-applications/
- Arduino. (2024). Arduino IDE (Version 2.3) [Computer software]. https://www.arduino.cc/en/software
- AtlasRFIDstore. (2019). RFID beginner's guide. https://www.atlasrfidstore.com/rfid-resources/rfid-beginners-guide/
- Blynk. (2024). Blynk IoT platform [Computer software]. https://blynk.io/
- CircuitDigest. (2022, February 11). Everything you need to know about the FlySky FS-i6 transmitter and receiver for drone control. https://circuitdigest.com/article/everything-you-need-to-know-about-the-flysky-fs-i6-transmitter-and-receiver-for-drone-control
- Dassault Systèmes. (2023). SOLIDWORKS [Computer software]. https://www.solidworks.com/
- Hübschmann, I. (2020, August 28). ESP32 for IoT: A complete guide. Nabto. https://www.nabto.com/guide-to-iot-esp-32/
- JetBrains. (2024). PyCharm (Version 2024.1) [Computer software]. https://www.jetbrains.com/pycharm/
- Jocher, G., Chaurasia, A., & Qiu, J. (2023). YOLO by Ultralytics (Version 8.0.0) [Computer software]. https://github.com/ultralytics/ultralytics
- Open Robotics. (2024). Robot Operating System (ROS 2) [Computer software]. https://www.ros.org/
- Santos, S. (2022, April 27). Arduino with load cell and HX711 amplifier (digital scale). Random Nerd Tutorials. https://randomnerdtutorials.com/arduino-load-cell-hx711/